

РЕФЕРАТ

КЛЮЧЕВЫЕ СЛОВА: АВТОМАТИЗИРОВАННОЕ РАБОЧЕЕ МЕСТО, КАРТОЧКА РОЗЫСКА, РЕЛЯЦИОННАЯ БАЗА ДАННЫХ, ПРОГРАММНЫЙ КОМПЛЕКС, СКВОЗНОЙ АУДИТ, ИНФОРМАЦИОННАЯ БЕЗОПАСНОСТЬ, АВТОМАТИЗАЦИЯ ДОКУМЕНТООБОРОТА, СИСТЕМНЫЙ АНАЛИЗ, ОПТИМИЗАЦИЯ ПРОЦЕССОВ.

Работа посвящена разработке автоматизированного рабочего места сотрудника отдела дознания военной комендатуры для учёта и сопровождения карточек розыска военнослужащих.

Актуальность исследования обусловлена необходимостью устранения критических недостатков ручного учёта: дублирования записей, потери версий документов, отсутствия сквозного аудита изменений и рисков несанкционированного доступа к служебным данным.

Объектом исследования выступает процессуальная деятельность отдела дознания.

Предметом исследования – программный комплекс, автоматизирующий создание, редактирование, поиск и аудит карточек розыска.

Цель работы – разработка программного продукта, обеспечивающего высокую скорость доступа к данным и минимизацию человеческого фактора.

В работе использованы методы системного анализа, объектно-ориентированного проектирования, нормализации данных, применения архитектурных паттернов, итеративной разработки и комплексного тестирования.

Практическая значимость определяется готовностью продукта к внедрению в изолированном контуре гарнизона. Разработанный программный комплекс снижает трудозатраты на 40–60 %, исключает ошибки ввода и соответствует требованиям законодательства в области защиты информации.

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей ВКР применяют следующие термины с соответствующими определениями.

API	Application Programming Interface — программный интерфейс приложения, набор спецификаций, позволяющий разным программным компонентам взаимодействовать между собой.
MVC	Model-View-Controller — архитектурный паттерн разделения данных (Model), пользовательского интерфейса (View) и логики управления (Controller).
UI/UX	User Interface / User Experience — пользовательский интерфейс и пользовательский опыт, совокупность визуальных и интерактивных аспектов взаимодействия человека с программным продуктом.
АРМ	Автоматизированное рабочее место — комплекс программно-аппаратных средств, предназначенный для автоматизации профессиональной деятельности конкретного специалиста.
КРД	Карточка розыска военнослужащего — основной учетный документ, содержащий социально-демографические данные, сведения о месте службы, эпизодах самовольного оставления части и процессуальных документах.
СОЧ	Самовольное оставление части — воинское правонарушение, заключающееся в отсутствии военнослужащего в месте службы без уважительных причин.

1 Анализ предметной области.....	7
1.1 Деятельность организации.....	7
1.2 Анализ аналогичных программных продуктов.....	10
1.3 Требования к разрабатываемому программному продукту.....	13
Технические требования	16
Требования информационной безопасности.....	17
2 Проектирование программного продукта.....	18
2.2 Выбор средств разработки программного продукта.....	18
2.2 Модель жизненного цикла программного продукта.....	20
2.4 Проектирование базы данных и интерфейса.....	32
3. Программная реализация	49
3.1 Реализация ключевых модулей программного продукта	50
3.2 Реализация пользовательского интерфейса	55
3.3 Тестирование и отладка программного продукта.....	62
4. Анализ качества и перспективы дальнейшего развития разработанного программного продукта.....	67
4.1 Анализ качества программного продукта	67
4.2 Перспективы развития программного продукта.....	72
ЗАКЛЮЧЕНИЕ	76
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	81

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящей ВКР применяют следующие сокращения и обозначения:

АРМ – Автоматизированное рабочее место

БД – База данных

ГИБДД – Государственная инспекция безопасности дорожного движения

ГОСТ – Государственный стандарт

ЗАГС – Орган регистрации актов гражданского состояния

КРД – Карточка розыска военнослужащего

ОМВД – Отдел Министерства внутренних дел

ПК – Персональный компьютер

СОЧ – Самовольное оставление части

СУБД – Система управления базами данных

ФЗ – Федеральный закон Российской Федерации

ФССП – Федеральная служба судебных приставов

ФСТЭК – Федеральная служба по техническому и экспортному контролю

ACID – Atomicity, Consistency, Isolation, Durability – свойства транзакций

БД

API – Application Programming Interface – интерфейс программирования приложений

BYTEA – Binary Data Type – тип данных PostgreSQL для хранения бинарных объектов

CRUD – Create, Read, Update, Delete – базовые операции работы с данными

FK – Foreign Key – внешний ключ в реляционной базе данных

GUI – Graphical User Interface – графический интерфейс пользователя

HTTP – HyperText Transfer Protocol – протокол передачи гипертекста

IDE – Integrated Development Environment – интегрированная среда разработки

JSON – JavaScript Object Notation – формат обмена данными

ВВЕДЕНИЕ

Актуальность исследования обусловлена необходимостью цифровой трансформации процессуальной деятельности подразделений военной юстиции. В настоящее время учёт карточек розыска КРД военнослужащих, самовольно оставивших воинскую часть, ведётся преимущественно в разрозненных электронных таблицах и бумажных журналах. Подобный подход приводит к дублированию записей, потере версий документов, отсутствию механизма версионирования и аудита изменений, а также создаёт риски несанкционированного доступа к персональным и служебным данным. Внедрение специализированного автоматизированного рабочего места АРМ позволяет централизовать хранение информации, автоматизировать генерацию процессуальных документов по утверждённым шаблонам.

Объектом исследования является процессуальная деятельность сотрудника отдела дознания военной комендатуры по учёту, обработке данных и документальному сопровождению карточек розыска военнослужащих.

Предметом исследования выступает программный комплекс АРМ, автоматизирующий создание, редактирование, поиск, экспорт отчётности и аудит действий при работе с КРД.

Цель работы – разработка и программная реализация настольного приложения для автоматизации полного цикла обработки карточек розыска, обеспечивающего высокую скорость доступа к данным, прозрачность процессуальных действий и минимизацию человеческого фактора при формировании официальных документов.

Для достижения поставленной цели были сформулированы и решены следующие задачи:

1. Изучить нормативно-правовую базу и организацию деятельности отдела дознания.

2. Сформулировать общие, функциональные, дизайнерские и технические требования к разрабатываемому программному продукту.
3. Спроектировать реляционную структуру базы данных.
4. Выбрать стек технологий для реализации программного продукта.
5. Реализовать модульную архитектуру приложения.
6. Провести комплексное функциональное, интеграционное, нагрузочное и security-тестирование.

Методы исследования: системный анализ предметной области, объектно-ориентированное проектирование, нормализация реляционных баз данных 3НФ, применение архитектурного паттерна Model-View-Controller MVC, итеративная разработка с элементами гибкого управления, функциональное и интеграционное тестирование, анализ производительности и безопасности.

Научная новизна работы заключается в разработке алгоритма динамического сопоставления переменных шаблонов процессуальных документов со столбцами базы данных, что позволяет гибко изменять структуру отчётов без модификации схемы БД. Также предложена реализация механизма полного аудита с сохранением старых и новых состояний полей, обеспечивающая полную ретроспективную трассируемость любых изменений в карточках розыска.

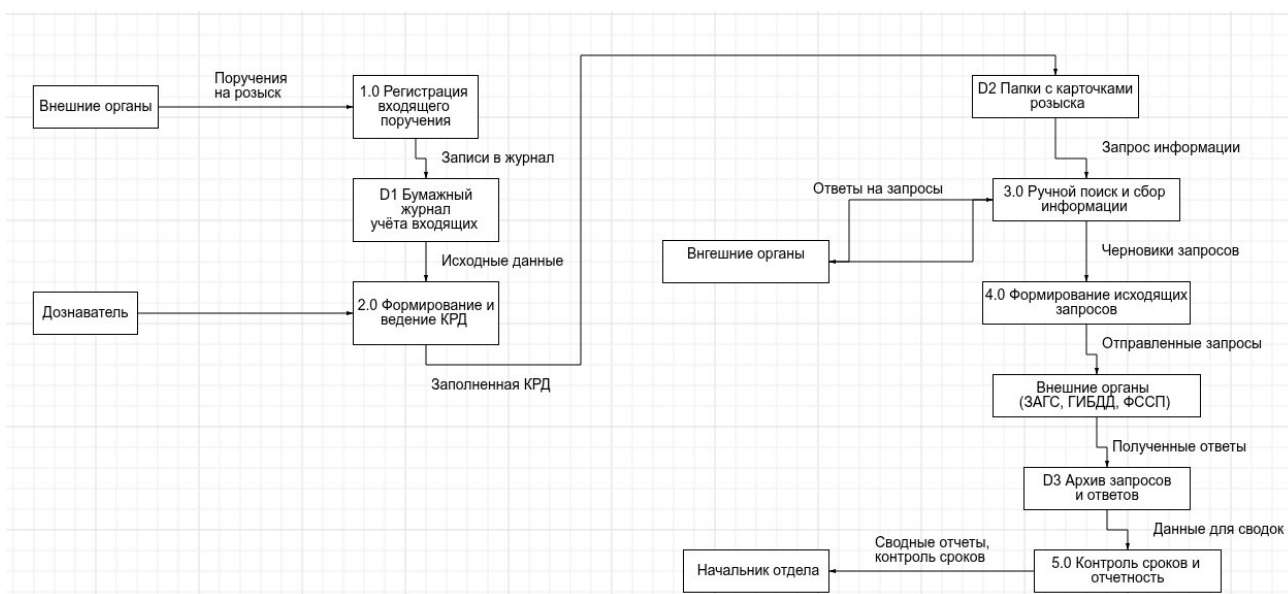
Практическая значимость в том, что разработанная АРМ снижает трудозатраты на регистрацию и обработку КРД на 40–60 %, исключает ошибки ручного ввода, а также поддерживает развёртывание на отечественных операционных системах в соответствии с политикой импортозамещения.

Общий объём работы составляет 68 страниц машинописного текста, включает 12 рисунков, 4 таблицы, 3 приложения. Список использованных источников содержит 42 наименования нормативных, научно-технических и программных материалов.

1 Анализ предметной области

1.1 Деятельность организации

Объектом исследования выступает подразделение военной комендатуры, осуществляющее процессуальную деятельность по учёту военнослужащих, самовольно оставивших воинскую часть СОЧ, а также обработку входящих и исходящих поручений на розыск. Деятельность строго регламентирована Уголовно-процессуальным кодексом Российской Федерации, Федеральным законом «О военной полиции и органах военной юстиции в Вооружённых Силах Российской Федерации», ведомственными приказами Министерства обороны РФ и внутренними инструкциями по делопроизводству гарнизона. Организационная структура подразделения предполагает чёткое распределение ролей: начальник отдела координирует процесс, дознаватели ведут непосредственную работу с материалами, оперативные дежурные обеспечивают первичную регистрацию входящих документов, а специалисты технической поддержки сопровождают учётные системы.



Рисунот 1.1 - Диаграмма поток данных до автоматизации

На рисунке 1.1 представлена диаграмма потоков данных DFD уровня 1, отражающая процесс обработки карточек розыска до внедрения АРМ. Внешними

субъектами выступают военные комиссариаты, органы внутренних дел, прокуратура и внешние ведомства. Процесс 1.0 обеспечивает ручную регистрацию входящих поручений с занесением данных в бумажный журнал учёта D1. Далее дознаватель вручную формирует КРД(Процесс 2.0, которые хранятся в бумажных папках D2. При необходимости уточнения данных иницируется ручной поиск Процесс 3.0 и подготовка бумажных запросов Процесс 4.0. Полученные ответы архивируются D3, а на основе накопленных данных вручную формируются сводные отчёты для контроля сроков Процесс 5.0, которые передаются начальнику отдела. Отсутствие автоматизации приводит к дублированию ввода, потере версий и нарушению процессуальных сроков, что обосновывает необходимость разработки специализированного АРМ.

Отсутствие единой информационной среды приводит к следующим критическим проблемам:

1. Дублирование записей и версионный хаос. При совместной работе нескольких дознавателей создаются копии файлов с идентичными именами, что приводит к конфликтам данных, утрате актуальной версии документа и невозможности оперативного определения статуса обработки карточки розыска.

2. Потеря данных и нарушение процессуальных сроков. Хранение материалов в локальных папках без централизованного резервного копирования создаёт риски безвозвратной утраты информации при аппаратных сбоях, вирусных атаках или некорректных действиях персонала. Отсутствие автоматических напоминаний о контрольных датах приводит к нарушению регламентных сроков розыскных мероприятий.

3. Отсутствие механизма версионирования и аудита изменений. В существующей модели невозможно отследить, кто, когда и какие изменения внёс в карточку розыска или шаблон процессуального документа. Это нарушает требования подотчётности, усложняет служебные проверки и делает

невозможным восстановление предыдущих состояний записей при выявлении ошибок ввода.

4. Риски несанкционированного доступа. Разрозненное хранение персональных данных военнослужащих в нешифрованных файлах на рабочих станциях не обеспечивает соответствия требованиям Федерального закона № 152-ФЗ «О персональных данных» и приказам ФСТЭК России. Отсутствие ролевой модели доступа и журналирования действий создаёт уязвимости для внутренних утечек конфиденциальной информации.

5. Низкая производительность рутинных операций. Ручной ввод данных, форматирование документов в текстовых редакторах, выгрузка отчётов методом «копировать-вставить» увеличивают трудозатраты на 40–60 %. Человеческий фактор становится причиной арифметических и орфографических ошибок, что недопустимо при формировании процессуальных документов, имеющих юридическую силу.

Таблица 1 – Функции подразделения и типы обрабатываемых данных

№	Функция подразделения	Тип обрабатываемых данных	Формат хранения
1	Регистрация входящих поручений	Реквизиты документов, даты, инициаторы	Бумажные журналы / локальные таблицы
2	Ведение карточек розыска (КРД)	Социально-демографические данные, фото, паспортные реквизиты	Разрозненные файлы Word/Excel, бумажные папки
3	Формирование исходящих запросов	Шаблоны запросов в ЗАГС, ГИБДД, ФССП, подписи должностных лиц	Отдельные .docx файлы без версионности
4	Учёт эпизодов СОЧ и адресов	Даты, места, свидетели, обстоятельства розыска	Текстовые описания в неструктурированном виде
5	Контроль сроков и отчётность	Даты поступления/ответа, статусы исполнения, статистика	Ручной сбор данных, сводные таблицы с ошибками

Вывод

Проведённый анализ деятельности отдела дознания выявил системные недостатки существующей модели документооборота. Разрозненное хранение информации, отсутствие единого источника истины, механизмов аудита и ролевого доступа делает процессуальную деятельность уязвимой к ошибкам, нарушениям сроков и рискам утечки персональных данных.

1.2 Анализ аналогичных программных продуктов

Проведён сравнительный анализ существующих программных решений, применяемых для автоматизации документооборота и учёта данных в правоохранительных и военных структурах. Рассмотрены как коммерческие системы управления предприятием и документооборотом, так и отраслевые информационные системы закрытого типа, а также распространённые ручные методы учёта.

На современном рынке программного обеспечения представлено множество систем для автоматизации документооборота и учёта данных. Однако специфика деятельности отделов дознания военных комендатур накладывает особые требования к программным продуктам, которые не всегда могут быть удовлетворены готовыми решениями.

Рассмотрим основные категории программных продуктов, которые потенциально могут быть использованы для автоматизации учёта карточек розыска военнослужащих:

К данной категории относятся такие системы как 1С:Документооборот, Directum RX. Эти решения представляют собой комплексные системы электронного документооборота СЭД с маршрутизацией документов, согласованием и контролем исполнения.

1С:Документооборот – это масштабная корпоративная система, обладающая широким функционалом для управления документами организации. Система поддерживает маршрутизацию документов, электронную подпись и

интеграцию с другими продуктами экосистемы 1С. Однако для задач учёта КРД данная платформа имеет ряд существенных ограничений:

- Высокая совокупная стоимость владения, включающая стоимость лицензий сервера и клиентских мест, а также необходимость привлечения сертифицированных специалистов для настройки и сопровождения, что экономически нецелесообразно для небольших подразделений военной комендатуры;

- Проприетарная архитектура платформы и метаданных, которая существенно усложняет внедрение специализированных алгоритмов динамического сопоставления полей шаблонов и гибкой обработки бинарных данных без глубокой кастомизации и привлечения разработчиков 1С;

- Избыточный функционал, ориентированный на крупные коммерческие и государственные организации, что приводит к перегруженности интерфейса и требует значительных затрат на адаптацию под узкоспециализированные процессуальные задачи дознания;

- Сложность развёртывания и обновления платформы в условиях полностью изолированного военного контура с ограниченным доступом к внешним сетям и репозиториям;

- Жёсткая привязка к внутренней модели данных платформы, которая не позволяет напрямую использовать оптимизированные структуры реляционных СУБД для маппинга или эффективного хранения больших бинарных объектов без применения дополнительных механизмов интеграции.

Directum RX также представляют собой мощные платформы документооборота. Они поддерживают гибкие бизнес-процессы, электронную цифровую подпись, аудит действий пользователей. Однако данные системы требуют выделенных серверов приложений, сложной настройки ролевой модели и не ориентированы на работу с фотоматериалами и специализированными сущностями типа КРД.

Наиболее распространённым подходом в настоящее время является использование офисных пакетов Microsoft Office (Excel, Word) в сочетании с бумажными журналами учёта.

Несмотря на кажущуюся простоту и доступность, ручной учёт создаёт критические риски: потерю данных, нарушение процессуальных сроков, невозможность оперативного поиска по комплексным критериям, отсутствие механизма массового экспорта отчётности. Кроме того, хранение фотоматериалов и сканов документов в разрозненных папках усложняет их привязку к конкретным записям и нарушает требования к целостности архива.

Таблица 2 – Матрица сравнения программных продуктов по ключевым критериям

Критерий сравнения	1С:Документооборот	Directum RX / Элан	Excel + Word	Разрабатываемое АРМ
Работа в изолированном контуре	3	3	1	5
Генерация и выгрузка отчётности	4	4	2	5
Гибкая настройка шаблонов	2	3	5	5
Кроссплатформенность и импортозамещение	3	3	5	5
Ролевая модель доступа	5	4	1	5
Сквозное журналирование (аудит)	4	4	1	5
Механизм восстановления данных	4	3	1	5
Стоимость владения и развёртывания	2	2	5	5

Анализ матрицы показывает, что разрабатываемое АРМ получает максимальные баллы по всем критическим параметрам, адаптированным под специфику военной юстиции. Корпоративные СЭД обладают развитыми

средствами аудита, но проигрывают по гибкости архитектуры и стоимости. Ручные методы формально выигрывают по скорости старта и кроссплатформенности, но не обеспечивают ни информационной безопасности, ни целостности данных. Ведомственные системы обладают высокой защищённостью, но полностью закрыты для интеграции.

Выводы

Выполнен сравнительный анализ существующих программных продуктов и ручных методов учёта. Подтверждена невозможность использования готовых коммерческих, открытых или ведомственных решений из-за узкой специфики работы с карточками розыска военнослужащих, отсутствия поддержки динамической настройки шаблонов, работы с фотоматериалами в централизованном формате и требований к изолированному контуру развёртывания.

1.3 Требования к разрабатываемому программному продукту

На основе проведённого анализа деятельности отдела дознания военной комендатуры, выявления информационных потоков и результатов сравнительного обзора существующих решений, сформулированы комплексные требования к автоматизированному рабочему месту АРМ сотрудника дознания.

Требования структурированы по пяти ключевым категориям: общие, функциональные, требования к пользовательскому интерфейсу (UI/UX), технические и требования информационной безопасности.

Общие требования

Программный продукт должен обеспечивать кроссплатформенность и совместимость с операционными системами Windows 10/11 (64-bit) и Linux включая сертифицированные отечественные ОС Astra Linux Special Edition и РЕД ОС. Обязательным является обеспечение информационной безопасности, система должна поддерживать хеширование паролей пользователей, разграничение прав доступа на уровне ролей защиту от brute-force атак и

контроль активных сессий. Система должна обеспечивать отказоустойчивость транзакций, поддержку резервного копирования базы данных, механизм мягкого удаления записей с возможностью восстановления из «Корзины» и полную независимость от внешних облачных сервисов, работая исключительно в локальной сети.

Функциональные требования

Программный комплекс должен обеспечивать надёжное управление доступом и аутентификацию пользователей, включая механизм проверки подлинности учётных данных с применением современных криптографических методов защиты. Система обязана поддерживать ролевую модель разграничения прав, которая гарантирует дифференцированный доступ к функциональным модулям строго в соответствии с должностными обязанностями сотрудников, а также предусматривать контроль активных сессий с возможностью их принудительного завершения и обязательным журналированием всех событий входа и выхода из системы.

В части управления данными предметной области комплекс должен поддерживать полный цикл операций создания, чтения, обновления и логического удаления записей основной сущности карточки розыска, а также всех связанных с ней объектов. Целостность информации обеспечивается за счёт механизмов транзакционной обработки и каскадного управления связями между таблицами базы данных. Система также должна предусматривать поддержку версионности и возможность восстановления ранее удалённых записей в рамках утверждённой политики хранения данных.

Для повышения эффективности работы оператора программный продукт должен предоставлять интеллектуальную поддержку ввода данных, реализуемую посредством контекстного автодополнения текстовых полей на основе анализа исторических данных системы с механизмом кэширования для оптимизации производительности. Одновременно с этим система обязана осуществлять

строгую валидацию вводимых значений на соответствие бизнес-правилам и установленным форматам, предоставляя пользователю структурированную обратную связь при обнаружении отклонений.

Автоматизация документооборота предполагает генерацию процессуальных документов на базе утверждённых шаблонов с динамической подстановкой актуальных данных из централизованного хранилища. Комплекс должен поддерживать конфигурируемые правила сопоставления переменных шаблона с атрибутами предметной области, включая возможность обработки составных полей, а все сгенерированные документы подлежат сохранению в едином хранилище с обеспечением возможности их последующего поиска и извлечения.

Модуль экспорта и формирования отчётности должен обеспечивать массовую выгрузку данных в универсальные табличные форматы с поддержкой гибкой настройки структуры выходных документов. Система обязана осуществлять адаптивное форматирование отчётов в строгом соответствии с корпоративными стандартами оформления, что гарантирует единообразие и юридическую значимость создаваемых материалов.

Важным архитектурным требованием является организация сквозного журналирования всех операций. Система должна фиксировать каждое значимое действие пользователя в защищённом журнале событий с обязательной привязкой к уникальным идентификаторам субъекта и объекта операции. Для обеспечения возможности ретроспективного анализа и аудита изменений требуется сохранять дифференциальные состояния данных в структурированном формате, позволяющем отслеживать историю модификаций на протяжении всего жизненного цикла записи.

Требования к дизайну UI/UX

Интерфейс пользователя должен быть построен на основе табличных представлений с поддержкой динамического поиска, сортировки по столбцам.

Форма детальной карточки КРД реализуется через вкладочную навигацию, обеспечивающую логическое разделение данных по категориям социально-демографические данные, адреса, места службы, СОЧ, запросы. Обязательна клиентская валидация обязательных полей перед сохранением. Цветовая индикация статусов записей, красный для «В розыске», зелёный для «Разыскан», и контекстные подсказки к столбцам БД улучшают читаемость интерфейса. Заккрытие окна с несохранёнными изменениями вызывает диалог подтверждения для предотвращения потери данных.

Технические требования

Минимальные аппаратные требования: процессор 64-разрядной архитектуры с многоядерной, объём оперативной памяти, достаточный для кэширования часто используемых данных и комфортной работы с выборками свыше тысячи записей, свободное дисковое пространство для хранения временных файлов, шаблонов документов и локальных отчётов.

Подробные технические требования представлены в таблице 3.

Таблица 3 – Технические требования к операционной среде и оборудованию

Компонент	Требование	Абстрактная спецификация
ОС Клиента	Кроссплатформенность, импортозамещение	Современные ОС общего назначения, включая сертифицированные отечественные платформы
ОС Сервера БД	Стабильность, поддержка штатного резервного копирования	Серверные дистрибутивы с долгосрочной поддержкой (LTS)
Процессор	Параллельная обработка запросов	64-разрядная архитектура, многоядерная структура, тактовая частота от 2.0 ГГц
Оперативная память	Кэширование данных, работа с большими выборками	Не менее 4 ГБ (рекомендуется 8 ГБ для оптимальной производительности)
Дисковое пространство	Хранение временных файлов, шаблонов, отчётов, логов	Не менее 500 МБ свободного пространства
Сетевое взаимодействие	Изолированный контур, шифрование трафика	Локальная сеть (LAN), стандарты защищённого сетевого обмена (TLS)

Требования информационной безопасности

Информационная безопасность АРМ должна быть реализована на нескольких уровнях защиты данных и контроля доступа. На уровне аутентификации будут применяться криптографические алгоритмы одностороннего хеширования паролей с настраиваемой вычислительной сложностью. На уровне авторизации будет внедрена ролевая модель управления доступом, строго ограничивающая функционал административных модулей для соответствующей роли; базовая роль предоставляет доступ исключительно к операциям учёта и генерации отчётности в пределах активной сессии.

На уровне хранения данных необходимо реализовать механизм логического удаления: вместо физической очистки записи активируется соответствующий статусный флаг. Все значимые действия пользователей будут регистрироваться в централизованном журнале аудита с фиксацией предыдущих и новых состояний полей в структурированном формате. Транзакционная целостность будет обеспечиваться через механизмы фиксации и автоматического отката операций при возникновении системных ошибок, предотвращая частичное сохранение данных. Резервное копирование должно выполняться регламентно средствами СУБД. Передача данных между клиентским интерфейсом и сервером базы данных должна быть защищена протоколами шифрования.

Выводы

Сформулированы комплексные требования к разрабатываемому автоматизированному рабочему месту сотрудника отдела дознания, охватывающие все критические аспекты проектирования и эксплуатации системы. Требования структурированы по пяти ключевым направлениям: общие, функциональные, требования к пользовательскому интерфейсу, технические и требования информационной безопасности. Каждое требование прошло верификацию на соответствие нормативным документам и было адаптировано

под условия работы в изолированном военном контуре. Сформулированные требования образуют формализованную основу для последующего архитектурного проектирования, выбора технологического стека и проведения приёмочных испытаний, гарантируя, что итоговый программный продукт будет соответствовать эксплуатационным, юридическим и техническим стандартам подразделений военной юстиции.

Выводы по разделу 1

Проведён детальный анализ процессуальной деятельности отдела дознания военной комендатуры, выявлены ключевые информационные потоки при регистрации, сопровождении и архивировании карточек розыска военнослужащих. Установлено, что существующие методы учёта характеризуются высоким уровнем ручного труда, отсутствием единого источника истины, рисками потери данных и нарушением процессуальных сроков, что создаёт критическую необходимость во внедрении специализированного программного комплекса. Выполнен сравнительный анализ коммерческих, открытых и ведомственных информационных систем, подтверждена невозможность их прямого применения для задач военной комендатуры из-за высокой совокупной стоимости владения, закрытой проприетарной архитектуры, отсутствия поддержки изолированного контура, негибкой работы с бинарными объектами и невозможности быстрой адаптации под узкоспециализированные процессуальные шаблоны. На основе выявленных недостатков и нормативных ограничений сформулированы и структурированы полные требования к разрабатываемому АРМ, охватывающие общие, функциональные, дизайнерские, технические аспекты и вопросы информационной безопасности. Требования прошли экспертизу на непротиворечивость, полноту и тестируемость, что позволяет перейти к этапу архитектурного проектирования и выбору технологического стека с чётким пониманием целевых показателей качества и безопасности системы.

2 Проектирование программного продукта

2.1 Выбор средств разработки программного продукта

Обоснование выбора технологического стека является фундаментальным этапом проектирования, определяющим производительность, безопасность, стоимость владения и перспективы масштабирования программного продукта. При формировании набора инструментов учитывались требования кроссплатформенности, совместимости с российскими операционными системами..

В качестве языка программирования выбран Python 3.10+. Он обеспечивает высокую скорость прототипирования, строгую типизацию, развитую экосистему библиотек и кроссплатформенность. Синтаксис языка способствует написанию читаемого и поддерживаемого кода, что критично для академических и ведомственных проектов с длительным жизненным циклом.

Для создания графического интерфейса выбран фреймворк PyQt6, предоставляющий нативный рендеринг, мощные табличные компоненты, встроенную интеграцию с SQL-моделями и строгое соблюдение паттерна MVC.

Ядром слоя данных выбрана СУБД PostgreSQL 16. Она обеспечивает полную поддержку ACID-транзакций, расширенный тип JSONB для гибких конфигураций маппинга, тип BYTEA для хранения документов и фотографий, частичные индексы и встроенный механизм резервного копирования.

В качестве драйвера подключения используется QPSQL, интегрированный в Qt-фреймворк. Он обеспечивает минимальный оверхед, поддержку подготовленных запросов и автоматическое экранирование параметров, что полностью исключает уязвимости SQL-инъекций.

Для генерации процессуальных документов используется библиотека python-docx. Она позволяет парсить структуру `.docx` шаблонов, заменять переменные с сохранением исходных стилей и поддерживать таблицы с колонтитулами.

Для массового экспорта отчётности в формат .xlsx применяется `openpyxl`, обеспечивающая точное форматирование ячеек, объединение, вставку изображений из `BYTEA`-полей и автоматическую настройку ширины столбцов.

Безопасность аутентификации обеспечивается библиотекой `bcrypt`. Она реализует адаптивную сложность хеширования, устойчивость к перебору паролей и радужным таблицам, что полностью соответствует современным стандартам NIST.

Для работы с фотоматериалами военнослужащих применяется `Pillow`, позволяющая выполнять ресайз с сохранением пропорций, конвертацию `RGBA`→`RGB` и строгий контроль размера файла перед сохранением в БД (≤ 5 МБ).

Вывод

Таким образом, выбранный технологический стек полностью соответствует функциональным, техническим и эксплуатационным требованиям к автоматизированному рабочему месту сотрудника отдела дознания. Применение языка Python 3.10+ в связке с фреймворком PyQt6 обеспечивает кроссплатформенность, высокую отзывчивость графического интерфейса и Использование СУБД PostgreSQL 16 с драйвером QPSQL гарантирует надёжное хранение структурированных и бинарных данных, поддержку ACID-транзакций, гибкую конфигурацию через JSONB и защиту от SQL-инъекций за счёт механизма подготовленных запросов. Библиотеки `python-docx` и `openpyxl` позволяют автоматизировать формирование процессуальных документов и аналитической отчётности в строгом соответствии с утверждёнными шаблонами, а инструменты `bcrypt` и `Pillow` обеспечивают криптографическую защиту учётных данных и оптимизированную обработку фотографий с контролем объёма.

2.2 Модель жизненного цикла программного продукта

Выбор модели жизненного цикла ЖЦ программного продукта является критическим этапом проектирования, определяющим последовательность выполнения работ, распределение ресурсов, механизмы контроля качества и сроки выхода готового изделия. В современной инженерии программного обеспечения используются четыре базовые модели: каскадная Waterfall, спиральная Spiral, гибкая Agile/Scrum и итеративно-инкрементная Iterative-Incremental.

Ниже приведено подробное описание каждой модели с обоснованием выбора наиболее подходящей для данной разработки.

КАСКАДНАЯ МОДЕЛЬ

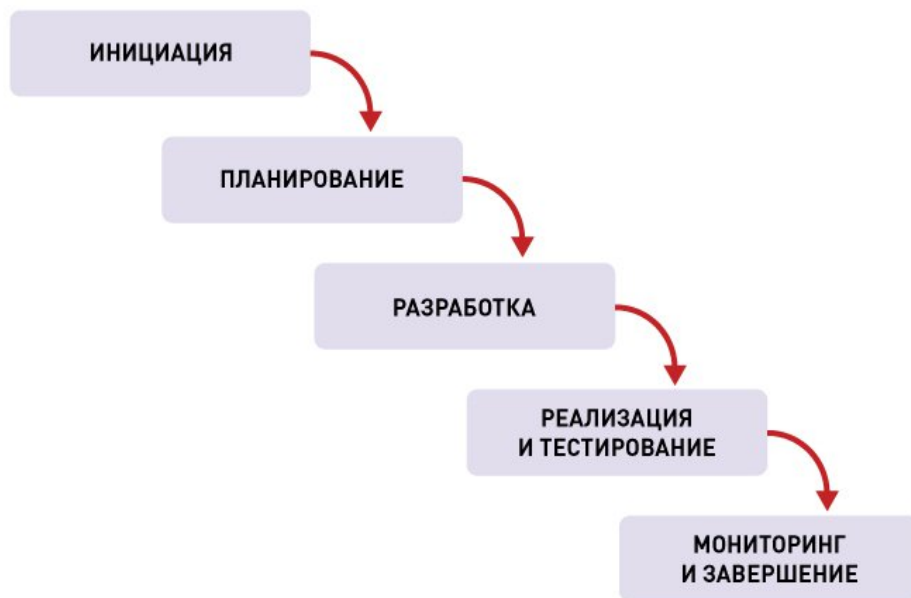


Рисунок 2.1 – Каскадная модель (Waterfall)

Каскадная модель Waterfall, показанная на рисунке 2.1, предполагает строго последовательное выполнение фаз разработки: сбор требований → проектирование → кодирование → тестирование → внедрение. Переход к следующему этапу возможен только после полного завершения предыдущего.

Данная модель обеспечивает высокую предсказуемость сроков и бюджета, а также чёткую документацию. Однако она не допускает корректировки требований на поздних стадиях, что критично для проектов, разрабатываемых в условиях меняющейся нормативной базы. Сфера применения ограничена проектами с изначально известными и неизменными требованиями.



Рисунок 2.2 - Спиральная модель Spiral

Спиральная модель Spiral, показанная на рисунке 2.2, фокусируется на управлении рисками через многократные циклы прототипирования, оценки рисков и разработки. Каждый виток спирали включает четыре фазы: определение целей, анализ рисков, разработку и планирование следующего этапа. Главное преимущество — минимизация технических рисков за счёт раннего выявления проблемных зон. Недостатком является высокая стоимость и сложность формализации метрик, что делает модель избыточной для настольного приложения с фиксированной предметной областью. Обычно применяется в крупных инновационных проектах.



Рисунок 2.3 – Итеративно-инкрементная модель

Итеративно-инкрементная модель, показанная на рисунке 2.3, объединяет преимущества предсказуемого планирования и адаптивной корректировки. Проект разбивается на последовательные этапы, по завершении каждого из которых формируется работоспособный инкремент программного продукта с расширенным функционалом. Преимущества: раннее тестирование ядра системы, прозрачность промежуточных результатов, возможность внесения изменений без полной переделки проекта.

На основе проведённого анализа для разработки АРМ сотрудника отдела дознания выбрана итеративно-инкрементная модель жизненного цикла с элементами гибкого управления проектами. Выбор обоснован следующими факторами:

- Поэтапное внедрение модулей позволяет тестировать критические функции на ранних стадиях, минимизируя риски интеграционных ошибок и обеспечивая стабильность ядра системы до добавления вспомогательных компонентов.

- Обратная связь от пользователя после каждой итерации обеспечивает коррекцию требований, интерфейса и логики валидации до финальной сборки, что снижает вероятность переделок на этапе приёмочных испытаний.

Вывод

В ходе сравнительного анализа моделей жизненного цикла программного обеспечения обоснован выбор итеративно-инкрементной модели с элементами гибкого управления для разработки автоматизированного рабочего места сотрудника отдела дознания. Рассмотренные альтернативные подходы не отвечают специфике проекта: каскадная модель излишне жёсткая и не допускает корректировки требований на поздних стадиях, а спиральная модель избыточна по сложности и стоимости для настольного приложения с фиксированной предметной областью. Выбранная модель обеспечивает поэтапную разработку и раннее тестирование критических модулей, позволяет оперативно адаптировать интерфейс и логику валидации на основе обратной связи, а также минимизирует риски интеграционных ошибок. Применение данного подхода гарантирует прозрачность процесса разработки, контроль качества на каждом этапе и полное соответствие разрабатываемого продукта актуальным нормативно-процессуальным требованиям.

2.3 Архитектура программного продукта

Проектирование архитектуры автоматизированного рабочего места АРМ сотрудника отдела дознания осуществлялось с учётом требований к отказоустойчивости, информационной безопасности, масштабируемости и удобства сопровождения. В качестве базовой парадигмы выбрана трёхуровневая клиент-серверная архитектура с адаптацией паттерна Model-View-Controller. Данное решение обеспечивает строгое разделение ответственности между слоями, минимизирует связность компонентов и упрощает независимое тестирование каждого модуля системы.

Трёхуровневая архитектура является стандартом де-факто для корпоративных информационных систем, обрабатывающих конфиденциальные данные. В рамках проекта архитектура разделена на три изолированных логических уровня: уровень представления, уровень бизнес-логики и уровень

данных. Взаимодействие между уровнями осуществляется через строго определённые интерфейсы, что исключает риски несанкционированного доступа и SQL-инъекций.

Общая схема архитектуры приложения представлена на рисунке 2.2.

Данная диаграмма демонстрирует трёхуровневую структуру системы с чётким разделением ответственности между компонентами.

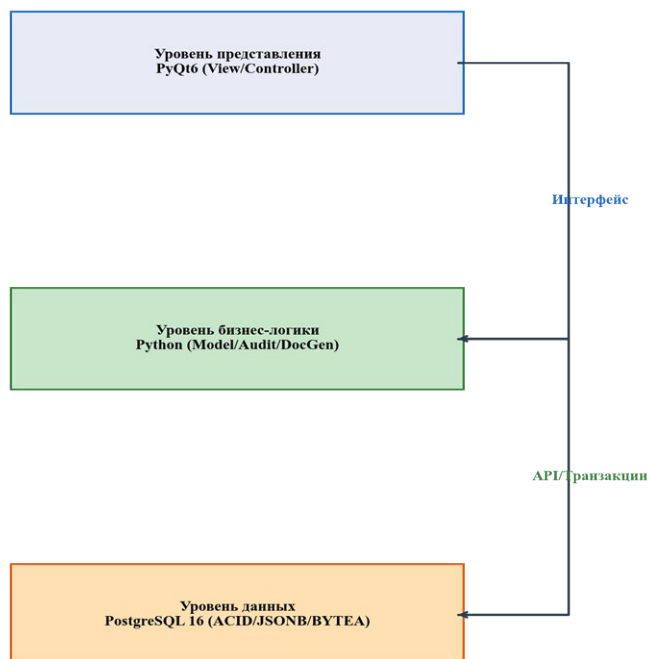


Рисунок 2.4 – Схема архитектуры приложения

На схеме представлены следующие компоненты системы:

- Уровень представления. Реализован на базе фреймворка PyQt6. Отвечает за отображение графического интерфейса пользователя, обработку событий и валидацию вводимых данных. Включает главные окна приложения, диалоговые формы и элементы управления.
- Уровень бизнес-логики. Реализован на языке Python. Содержит основные алгоритмы работы системы: генерацию процессуальных документов, экспорт данных в Excel, ведение журнала аудита, автодополнение полей и обработку бизнес-правил предметной области.

– Уровень данных. Реализован на базе СУБД PostgreSQL версии 16. Обеспечивает надёжное хранение данных с поддержкой ACID-транзакций, работу с бинарными данными для хранения фотографий и документов, использование гибких структур JSONB для конфигураций.

Диаграмма вариантов использования

Диаграмма вариантов использования, показанная на рисунке 2.5, определяет границы системы и роли пользователей. Данная диаграмма показывает, какие функции доступны различным категориям пользователей системы.

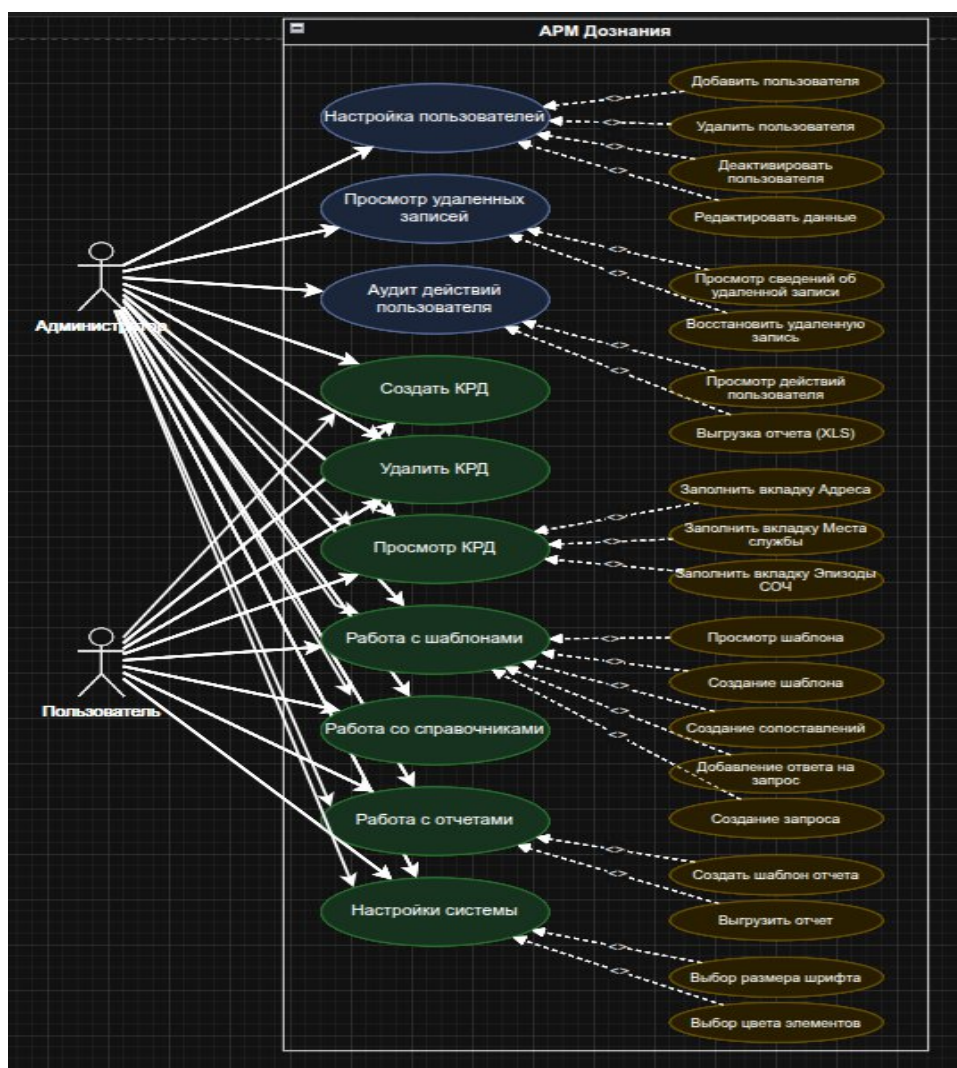


Рисунок 2.5 – Диаграмма вариантов использования системы

На диаграмме идентифицированы следующие ключевые элементы системы:

— Актор «Дознаватель». Выступает основным конечным пользователем системы, ответственным за ведение оперативно-розыскной деятельности. Его функционал включает полный жизненный цикл обработки карточки розыска: создание новых записей, внесение изменений в социально-демографические данные, адреса проживания и сведения о местах службы, а также фиксацию эпизодов самовольного оставления части. Помимо работы с данными, дознаватель инициирует процессы генерации процессуальных документов на основе утверждённых шаблонов и осуществляет выгрузку аналитических отчётов в табличном формате для предоставления руководству.

— Актор «Администратор». Представляет собой пользователя с расширенными полномочиями, обеспечивающего техническую и организационную поддержку работы системы. В его обязанности входит управление нормативно-справочной информацией, контроль активных пользовательских сессий и мониторинг целостности данных. Администратор также обладает правом доступа к журналу аудита действий и выполняет функции восстановления записей, подвергшихся процедуре мягкого удаления, из архива.

— Use Cases. Отражают основные сценарии взаимодействия с системой, сгруппированные по логическим модулям: управление реестром КРД, работа с документооборотом, администрирование справочников и конфигурация параметров отчётности.

— Механизм аутентификации и авторизации. Является обязательным предварительным условием для доступа к любому функционалу системы. Реализован на базе ролевой модели доступа с использованием криптографического хеширования паролей алгоритмом bcrypt, что исключает возможность несанкционированного входа и обеспечивает защиту учётных данных в соответствии с требованиями информационной безопасности.

Диаграмма активностей

Диаграмма активностей, показанная на рисунке 2.6, визуализирует алгоритм создания и сохранения новой карточки розыска. Данная диаграмма показывает последовательность действий системы и пользователя при добавлении новой записи.

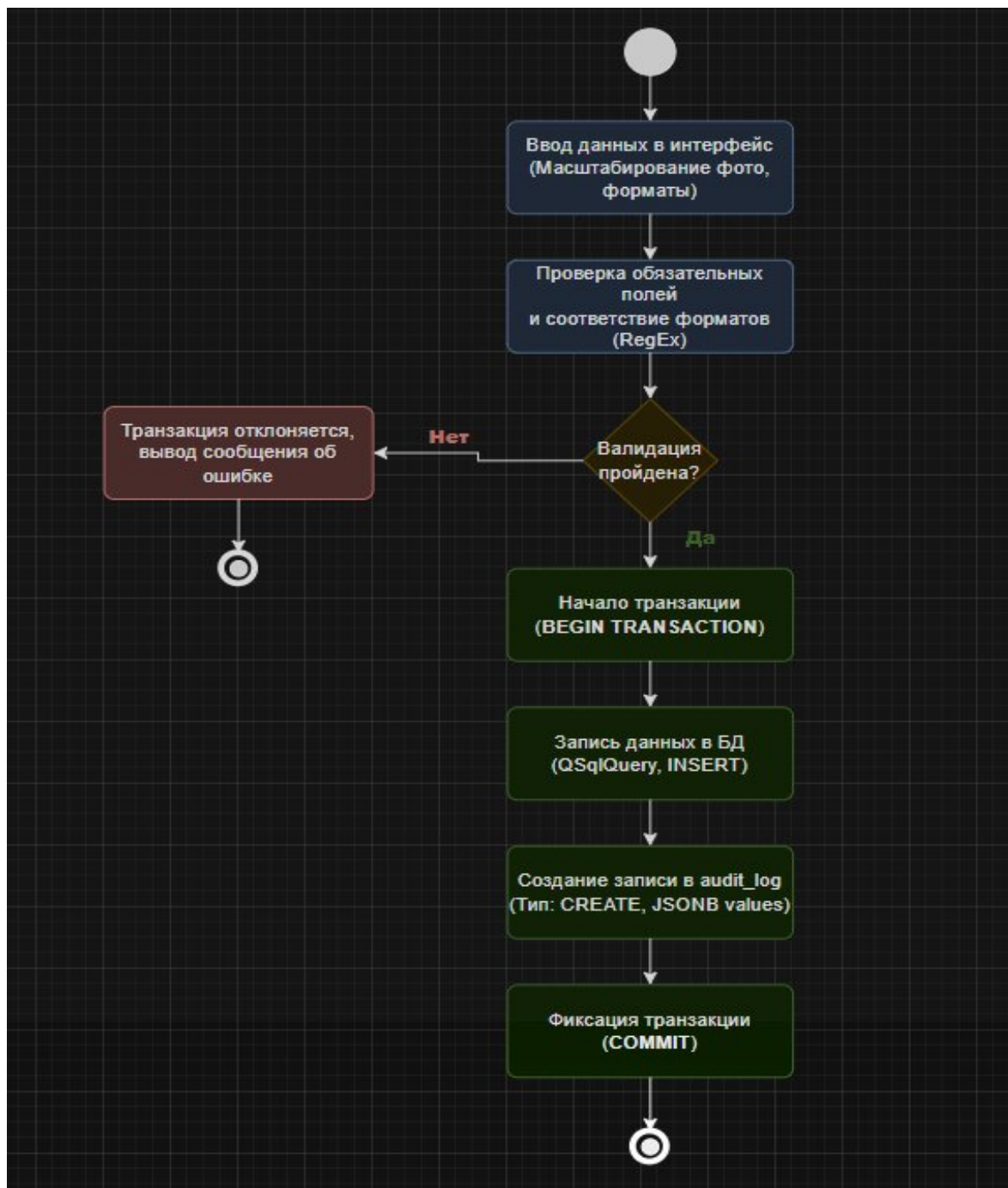


Рисунок 2.6 – Диаграмма активностей создания и сохранения КРД

Процесс создания КРД включает следующие этапы:

1. Начальное состояние. Процесс начинается с ввода данных в интерфейс системы. Пользователь заполняет форму карточки розыска, включая масштабирование фотографий и проверку форматов данных.
2. Проверка обязательных полей. Система выполняет автоматическую проверку заполнения обязательных полей и соответствие форматов регулярным выражениям. Проверяются форматы паспортных данных, военного билета, банковских карт.
3. Решение о валидации. Diamond-элемент принимает решение: если валидация не пройдена, транзакция отклоняется и пользователю выводится структурированное сообщение об ошибке. Если валидация пройдена, процесс продолжается.
4. Начало транзакции. При успешной валидации система начинает транзакцию базы данных для обеспечения атомарности операции.
5. Запись данных в БД. Данные карточки розыска записываются в таблицу `krd.social_data` с использованием подготовленных SQL-запросов через `QSqlQuery`.
6. Создание записи в `audit_log`. Одновременно с сохранением данных создаётся запись в таблице журнала аудита с типом действия `CREATE` и фиксацией новых значений в формате `JSONB`.
7. Фиксация транзакции. Транзакция завершается командой `COMMIT`, что гарантирует атомарность операции и отсутствие «висячих» состояний в базе данных.
8. Конечное состояние. Процесс завершается успешно, пользователю выводится сообщение о создании карточки розыска.

Диаграмма последовательности

Диаграмма последовательности, показанная на рисунке 2.7, детализирует временное взаимодействие слоёв MVC при сохранении записи. Данная диаграмма показывает порядок обмена сообщениями между компонентами системы во времени.

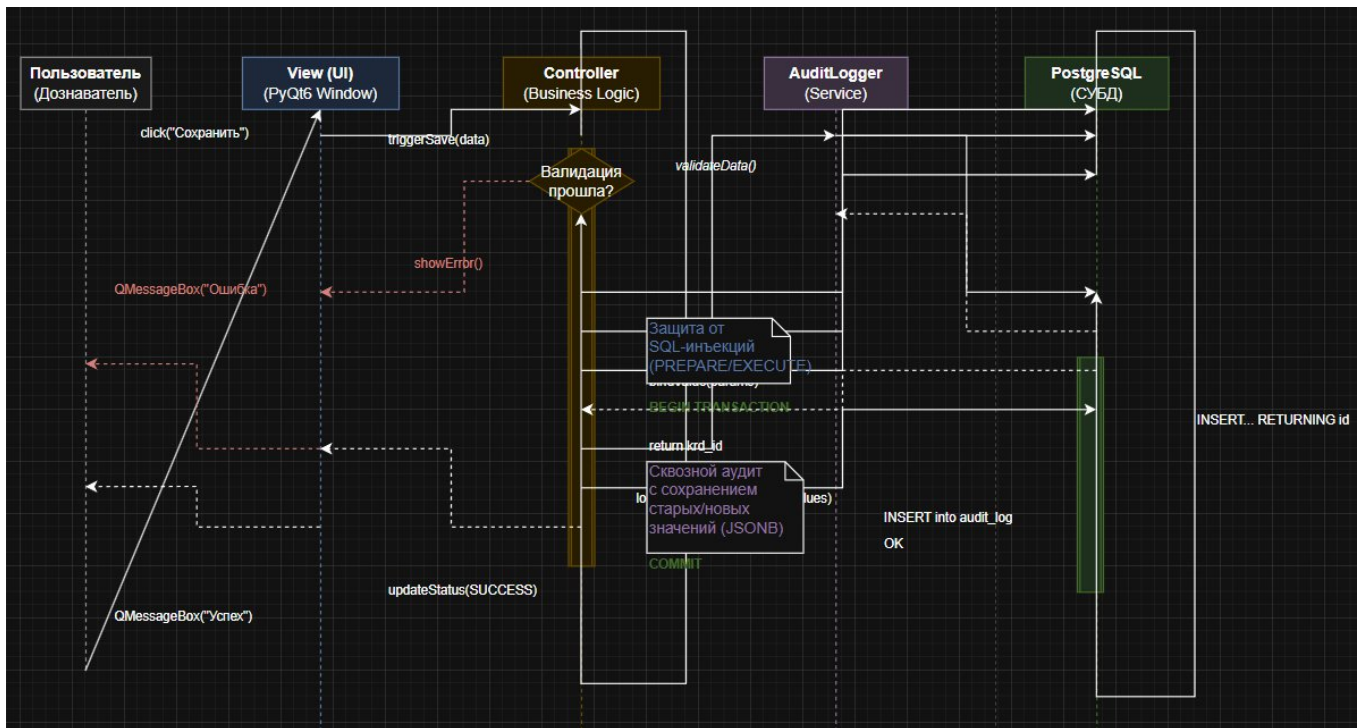


Рисунок 2.7 – Диаграмма последовательности сохранения данных

Последовательность взаимодействия компонентов:

1. Пользователь инициирует действие, нажимая кнопку «Сохранить» в интерфейсе.
2. View принимает событие и передаёт данные методу triggerSave контроллера.
3. Controller выполняет бизнес-логику валидации данных через метод validateData(). При ошибке вызывается showError() и пользователю выводится QMessageBox с сообщением об ошибке.

4. При успешной валидации контроллер подготавливает SQL-запрос через `PREPARE Query()` и передаёт параметры через `bindValue` для предотвращения SQL-инъекций.

5. PostgreSQL выполняет `INSERT` запрос и возвращает сгенерированный `id` записи через `RETURNING id`.

6. Controller вызывает метод логгера аудита `AuditLogger.logAction` для записи информации о созданной записи в журнал аудита.

7. `AuditLogger` → PostgreSQL выполняет `INSERT into audit_log` с сохранением diff-значений в формате JSONB.

8. Controller → PostgreSQL выполняет `COMMIT` для фиксации транзакции.

9. Controller → View возвращает результат `updateStatus` и выводит пользователю `QMessageBox` с сообщением об успешном сохранении.

При возникновении ошибки на любом этапе вызывается `ROLLBACK`, интерфейс остаётся в стабильном состоянии, а пользователю выводится сообщение с кодом исключения СУБД.

Архитектура системы реализует многоуровневую защиту данных в соответствии с требованиями Федерального закона № 152-ФЗ «О персональных данных» и приказами ФСТЭК России № 17 и 21.

— Аутентификация. Хеширование паролей алгоритмом `bcrypt` с настраиваемой вычислительной сложностью, контроль активных сессий, защита от brute-force атак.

— Авторизация. Ролевая модель разграничения доступа с чётким разделением прав: роль «Дознаватель» имеет доступ к операциям учёта и генерации отчётности, роль «Администратор» обладает расширенными правами управления системой.

— Целостность данных. ACID-транзакции обеспечивают атомарность, согласованность, изоляцию и долговечность операций. Механизм мягкого

удаления с флагом `is_deleted` позволяет восстанавливать ошибочно удалённые записи. Advisory Locks предотвращают конфликты при одновременном редактировании.

— Сквозной аудит. Все значимые действия пользователей регистрируются в журнале аудита с фиксацией предыдущих и новых состояний полей в формате JSONB, что обеспечивает полную ретроспективную трассируемость любых изменений.

Вывод

Разработанная трёхуровневая архитектура с адаптацией паттерна MVC обеспечивает чёткое разделение ответственности между модулями, что гарантирует высокую модульность, удобство сопровождения и независимое тестирование компонентов системы. Формализация бизнес-процессов посредством диаграмм вариантов использования, активностей и последовательности наглядно демонстрирует прозрачность взаимодействия слоёв и надёжность алгоритмов создания и сохранения карточек розыска. Внедрённые механизмы информационной безопасности, включая ролевую модель доступа, ACID-транзакции, мягкое удаление и сквозной аудит изменений в формате JSONB, полностью соответствуют требованиям законодательства о защите персональных данных.

2.4 Проектирование базы данных и интерфейса

Информационная модель системы спроектирована в виде реляционной базы данных PostgreSQL со схемой `krd`. Схема включает 26 таблиц, нормализованных до третьей нормальной формы. Нормализация исключила транзитивные зависимости: все неключевые атрибуты зависят исключительно от первичного ключа. Схема базы данных показана на рисунке 2.9

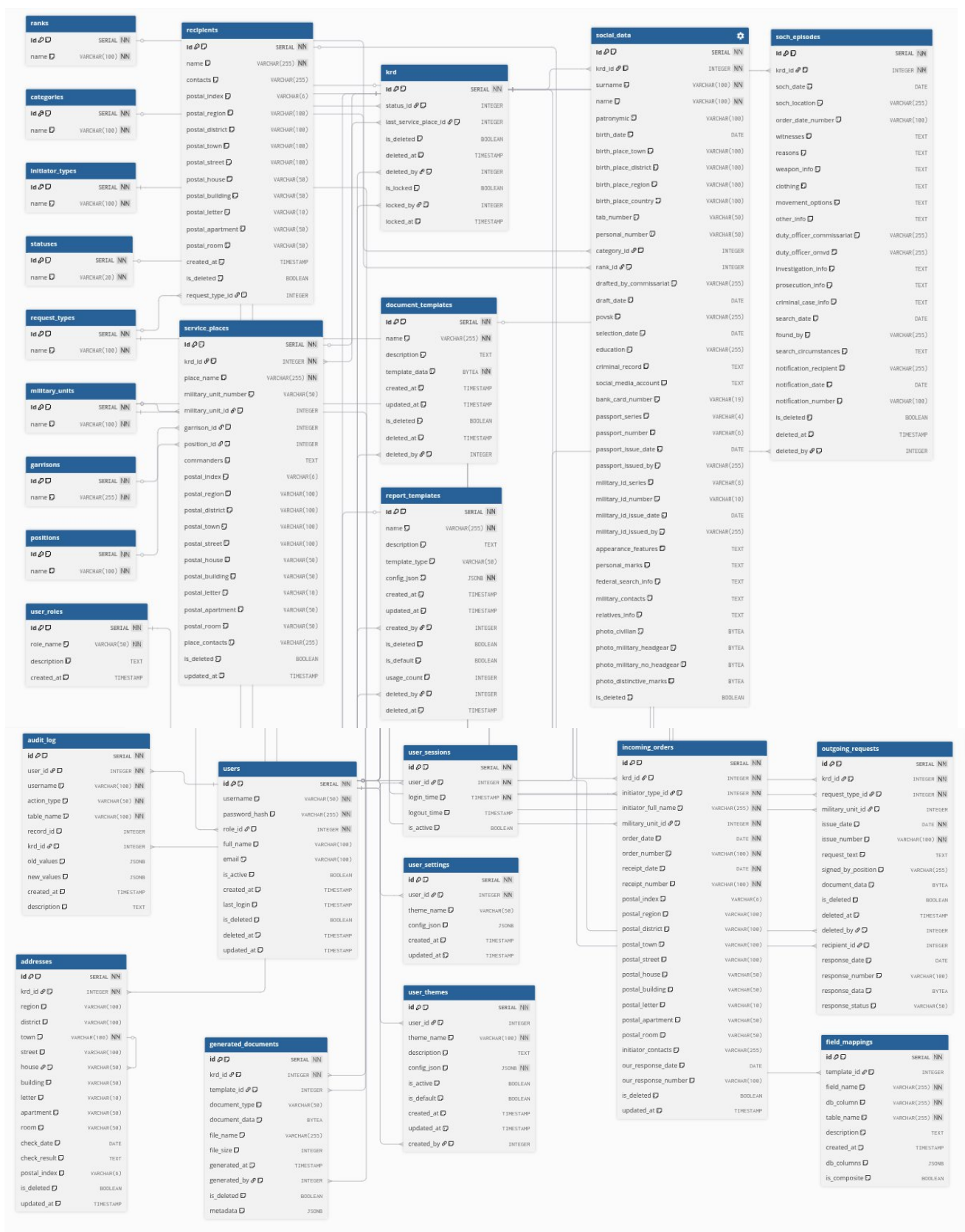


Рисунок 2.9 Схема базы данных

Центральная таблица схемы krd, представляющая карточку розыска военнослужащего. Является точкой привязки для всех связанных сущностей через поле krd_id. Хранит статус розыска, флаги мягкого удаления и блокировки для конкурентного доступа. Ее структура показана в таблице 4.

Таблица 4 – Структура таблицы krd.krd

Поле	Тип	Ограничения	Описание
id	SERIAL	PK	Уникальный ID карточки
status_id	INTEGER	FK → statuses, SET NULL	Статус (В розыске, Разыскан)
last_service_place_id	INTEGER	FK → service_places	ID последнего места службы
is_deleted	BOOLEAN	DEFAULT FALSE	Флаг мягкого удаления
deleted_at	TIMESTAMP	NULL	Время удаления
deleted_by	INTEGER	FK → users	Кто удалил
is_locked	BOOLEAN	DEFAULT FALSE	Advisory Lock флаг
locked_by	INTEGER	FK → users	Кто заблокировал
locked_at	TIMESTAMP	NULL	Время блокировки

Таблица социальные данные хранит персональные данные военнослужащего: ФИО, паспортные реквизиты, данные военного билета, место рождения, образование, судимость, банковские реквизиты и контакты. Содержит 4 поля BYTEA для хранения фотографий военнослужащего. Связана с krd через уникальное ограничение krd_id. Структура показана в таблице 5.

Таблица 5 – Структура таблицы krd.social_data

Поле	Тип	Ограничения	Описание
id	SERIAL	PK	Уникальный ID записи
krd_id	INTEGER	FK → krd.id, CASCADE, UNIQUE	Ссылка на КРД
surname	VARCHAR(100)	NOT NULL	Фамилия
name	VARCHAR(100)	NOT NULL	Имя
patronymic	VARCHAR(100)	NULL	Отчество
birth_date	DATE	NULL	Дата рождения
birth_place_town	VARCHAR(100)	NULL	Населённый пункт рождения

birth_place_district	VARCHAR(100)	NULL	Район рождения
birth_place_region	VARCHAR(100)	NULL	Субъект РФ
birth_place_country	VARCHAR(100)	NULL	Страна
tab_number	VARCHAR(50)	CHECK digits	Табельный номер
personal_number	VARCHAR(50)	CHECK digits	Личный номер
category_id	INTEGER	FK → categories, SET NULL	Категория в/с
rank_id	INTEGER	FK → ranks, SET NULL	Воинское звание
drafted_by_commissariat	VARCHAR(255)	NULL	Военкомат призыва
draft_date	DATE	NULL	Дата призыва
povsk	VARCHAR(255)	NULL	ПОВСК
selection_date	DATE	NULL	Дата отбора
education	VARCHAR(255)	NULL	Образование
criminal_record	TEXT	NULL	Судимость
social_media_account	TEXT	NULL	Аккаунты в соцсетях
bank_card_number	VARCHAR(19)	CHECK regex	Номер карты
passport_series	VARCHAR(4)	CHECK 4 digits	Серия паспорта
passport_number	VARCHAR(6)	CHECK 6 digits	Номер паспорта
passport_issue_date	DATE	NULL	Дата выдачи паспорта
passport_issued_by	VARCHAR(255)	NULL	Кем выдан
military_id_series	VARCHAR(8)	CHECK alphanum	Серия ВБ
military_id_number	VARCHAR(10)	CHECK digits	Номер ВБ
military_id_issue_date	DATE	NULL	Дата выдачи ВБ
military_id_issued_by	VARCHAR(255)	NULL	Кем выдан ВБ
appearance_features	TEXT	NULL	Особенности внешности
personal_marks	TEXT	NULL	Приметы (тату, шрамы)
federal_search_info	TEXT	NULL	Фед. розыск
military_contacts	TEXT	NULL	Контакты
relatives_info	TEXT	NULL	Родственники
photo_civilian	BYTEA	NULL	Фото в гражданском
photo_military_headgear	BYTEA	NULL	Фото в форме с фуражкой
photo_military_no_headgear	BYTEA	NULL	Фото в форме без фуражки
photo_distinctive_marks	BYTEA	NULL	Фото примет
is_deleted	BOOLEAN	DEFAULT FALSE	Мягкое удаление

Таблица krd.addresses хранит список адресов проживания военнослужащего, привязанных к карточке розыска. Каждая запись содержит

полную адресную структуру и результаты адресной проверки. Ее структура показана в таблице 6

Таблица 6 – Структура таблицы krd.addresses

Поле	Тип	Ограничения	Описание
id	SERIAL	PK	Уникальный ID
krd_id	INTEGER	FK → krd.id, CASCADE	Ссылка на КРД
region	VARCHAR(100)	NULL	Субъект РФ
district	VARCHAR(100)	NULL	Район
town	VARCHAR(100)	NOT NULL	Населённый пункт
street	VARCHAR(100)	NULL	Улица
house	VARCHAR(50)	NULL	Номер дома
building	VARCHAR(50)	NULL	Корпус
letter	VARCHAR(10)	NULL	Литера
apartment	VARCHAR(50)	NULL	Квартира
room	VARCHAR(50)	NULL	Комната
check_date	DATE	NULL	Дата проверки
check_result	TEXT	NULL	Результат
postal_index	VARCHAR(6)	CHECK 6 digits	Индекс
is_deleted	BOOLEAN	DEFAULT FALSE	Мягкое удаление

Таблица krd.service_places Фиксирует историю прохождения военнослужащим военной службы: наименования частей, воинские должности, гарнизоны, контакты командного состава и почтовые адреса мест дислокации. Ее структура указана в таблице 7.

Таблица 7 – Структура таблицы krd.service_places

Поле	Тип	Ограничения	Описание
id	SERIAL	PK	Уникальный ID
krd_id	INTEGER	FK → krd.id, CASCADE	Ссылка на КРД
place_name	VARCHAR(255)	NOT NULL	Наименование места службы
military_unit_number	VARCHAR(50)	NULL	Номер в/ч
military_unit_id	INTEGER	FK → military_units, SET NULL	Управление
garrison_id	INTEGER	FK → garrisons, SET	Гарнизон

		NULL	
position_id	INTEGER	FK → positions, SET NULL	Должность
commanders	TEXT	NULL	Командиры
postal_*	VARCHAR(50-100)	NULL	Почтовый адрес
place_contacts	VARCHAR(255)	NULL	Контакты
is_deleted	BOOLEAN	DEFAULT FALSE	Мягкое удаление

Таблица krd.incoming_orders регистрирует поручения на розыск, поступающие от внешних структур. Фиксирует реквизиты поручения, инициатора, почтовый адрес инициатора и ответы комендатуры. Ее структура показана в таблице 8.

Таблица 8 – Структура таблицы krd.incoming_orders

Поле	Тип	Ограничения	Описание
id	SERIAL	PK	Уникальный ID
krd_id	INTEGER	FK → krd.id, CASCADE	Ссылка на КРД
initiator_type_id	INTEGER	FK → initiator_types	Тип инициатора
initiator_full_name	VARCHAR(255)	NOT NULL	Инициатор
military_unit_id	INTEGER	FK → military_units	Управление
order_date	DATE	NOT NULL	Дата поручения
order_number	VARCHAR(100)	NOT NULL	Номер поручения
receipt_date	DATE	NOT NULL	Дата поступления
receipt_number	VARCHAR(100)	NOT NULL	Входящий номер
postal_*	VARCHAR(50-100)	NULL	Почтовый адрес
initiator_contacts	VARCHAR(255)	NULL	Контакты
our_response_date	DATE	NULL	Дата ответа
our_response_number	VARCHAR(100)	NULL	Номер ответа
is_deleted	BOOLEAN	DEFAULT FALSE	Мягкое удаление

Таблица krd.soch_episodes хранит информацию о каждом эпизоде самовольного оставления части: дата и место СОЧ, приказ, очевидцы, вероятные причины, описание одежды, сведения об оружии, контакты дежурных, результаты проверок и обстоятельства розыска. Ее структура показана в таблице 9.

Таблица 9 – Структура таблицы krd.soch_episodes

Поле	Тип	Ограничения	Описание
id	SERIAL	PK	Уникальный ID
krd_id	INTEGER	FK → krd.id, CASCADE	Ссылка на КРД
soch_date	DATE	NULL	Дата СОЧ
soch_location	VARCHAR(255)	NULL	Место СОЧ
order_date_number	VARCHAR(100)	NULL	Дата и номер приказа
witnesses	TEXT	NULL	Очевидцы
reasons	TEXT	NULL	Вероятные причины
weapon_info	TEXT	NULL	Оружие
clothing	TEXT	NULL	Описание одежды
movement_options	TEXT	NULL	Направления движения
other_info	TEXT	NULL	Другая информация
duty_officer_commissariat	VARCHAR(255)	NULL	Дежурный ВК
duty_officer_omvd	VARCHAR(255)	NULL	Дежурный ОМВД
investigation_info	TEXT	NULL	Проверка
prosecution_info	TEXT	NULL	Прокуратура
criminal_case_info	TEXT	NULL	Уголовное дело
search_date	DATE	NULL	Дата розыска
found_by	VARCHAR(255)	NULL	Кем разыскан
search_circumstances	TEXT	NULL	Обстоятельства
notification_recipient	VARCHAR(255)	NULL	Адресат уведомления
notification_date	DATE	NULL	Дата уведомления
notification_number	VARCHAR(100)	NULL	Номер уведомления
is_deleted	BOOLEAN	DEFAULT FALSE	Мягкое удаление

Таблица krd.outgoing_requests учитывает исходящие запросы в ЗАГС, ГИБДД, ФССП и другие органы. Содержит тип запроса, текст, подписанта, адресата, статус ответа и бинарные данные сгенерированных документов и полученных ответов. Ее структура показана в таблице 10.

Таблица 10 – Структура таблицы krd.outgoing_requests

Поле	Тип	Ограничения	Описание
id	SERIAL	PK	Уникальный ID
krd_id	INTEGER	FK → krd.id, CASCADE	Ссылка на КРД
request_type_id	INTEGER	FK → request_types	Тип запроса
military_unit_id	INTEGER	FK → military_units, SET NULL	Управление адресата
issue_date	DATE	NOT NULL	Дата запроса
issue_number	VARCHAR(100)	NOT NULL	Номер запроса
request_text	TEXT	NULL	Текст запроса
signed_by_position	VARCHAR(255)	NULL	Должность подписанта
document_data	BYTEA	NULL	Документ (.docx)
recipient_id	INTEGER	FK → recipients, SET NULL	Адресат
response_date	DATE	NULL	Дата ответа
response_number	VARCHAR(100)	NULL	Номер ответа
response_data	BYTEA	NULL	Документ ответа
response_status	VARCHAR(50)	DEFAULT Ожидание	Статус
is_deleted	BOOLEAN	DEFAULT FALSE	Мягкое удаление

Таблица krd.document_templates хранит шаблоны процессуальных документов в бинарном формате. Используется генератором документов для подстановки данных из БД в переменные {{field}}. Ее структура показана в таблице 11.

Таблица 11 – Структура таблицы krd.document_templates

Поле	Тип	Ограничения	Описание
id	SERIAL	PK	ID шаблона
name	VARCHAR(255)	NOT NULL	Название
description	TEXT	NULL	Описание
template_data	BYTEA	NOT NULL	Файл .docx
is_deleted	BOOLEAN	DEFAULT FALSE	Мягкое удаление
deleted_at	TIMESTAMP	NULL	Дата удаления
deleted_by	INTEGER	FK → users	Кто удалил

Таблица `krd.field_mappings` хранит конфигурацию связи переменных шаблона со столбцами БД. Поле `db_columns` позволяет описывать составные поля. Динамический маппинг исключает необходимость изменения схемы БД при добавлении новых шаблонов. Ее структура показана в таблице 12.

Таблица 12 – Структура таблицы `krd.field_mappings`

Поле	Тип	Ограничения	Описание
<code>id</code>	SERIAL	PK	ID маппинга
<code>template_id</code>	INTEGER	FK → <code>document_templates.id</code>	Ссылка на шаблон
<code>field_name</code>	VARCHAR(255)	NOT NULL	Переменная <code>{{field}}</code>
<code>db_column</code>	VARCHAR(255)	NOT NULL	Столбец БД
<code>table_name</code>	VARCHAR(255)	NOT NULL	Таблица БД
<code>db_columns</code>	JSONB	NULL	Конфигурация составных полей
<code>is_composite</code>	BOOLEAN	DEFAULT FALSE	Признак составного поля

Таблица `krd.users` хранит учётные записи сотрудников системы. Пароли хешируются алгоритмом `bcrypt`. Поддерживает мягкое удаление и контроль статуса. Ее структура показана в таблице 13.

Таблица 13 – Структура таблицы `krd.users`

Поле	Тип	Ограничения	Описание
<code>id</code>	SERIAL	PK	ID пользователя
<code>username</code>	VARCHAR(50)	UNIQUE, NOT NULL	Логин
<code>password_hash</code>	VARCHAR(255)	NOT NULL	<code>bcrypt</code> -хеш
<code>role_id</code>	INTEGER	FK → <code>user_roles.id</code>	Роль
<code>full_name</code>	VARCHAR(100)	NULL	ФИО
<code>email</code>	VARCHAR(100)	NULL	Email
<code>is_active</code>	BOOLEAN	DEFAULT TRUE	Активен
<code>is_deleted</code>	BOOLEAN	DEFAULT FALSE	Мягкое удаление

Таблица `krd.audit_log` фиксирует все действия пользователей. Сохраняет старые и новые значения полей в формате JSONB для полной ретроспективной трассируемости. Структура показана в таблице 14.

Таблица 14 – Структура таблицы krd.audit_log

Поле	Тип	Ограничения	Описание
id	SERIAL	PK	ID записи журнала
user_id	INTEGER	NOT NULL, FK → users	Кто выполнил
username	VARCHAR(100)	NOT NULL	Логин
action_type	VARCHAR(50)	NOT NULL	Тип действия
table_name	VARCHAR(100)	NOT NULL	Таблица
record_id	INTEGER	NULL	ID записи
krd_id	INTEGER	NULL	Привязка к КРД
old_values	JSONB	NULL	Старые значения
new_values	JSONB	NULL	Новые значения
description	TEXT	NULL	Описание

Справочники обеспечивают ссылочную целостность и унификацию данных. Содержат статические списки категорий, званий, статусов, воинских должностей, гарнизонов, военных управлений, типов запросов и инициаторов. Они показаны в таблице 15.

Таблица 15 – Справочные таблицы схемы krd

Таблица	Поля	Ограничения	Назначение
categories	id, name	PK, UNIQUE(name)	Категории военнослужащих
ranks	id, name	PK, UNIQUE(name)	Воинские звания
statuses	id, name	PK, UNIQUE(name)	Статусы КРД
military_units	id, name	PK, UNIQUE(name)	Военные управления
garrisons	id, name	PK, UNIQUE(name)	Гарнизоны
positions	id, name	PK, UNIQUE(name)	Воинские должности
request_types	id, name	PK, UNIQUE(name)	Типы запросов
initiator types	id, name	PK, UNIQUE(name)	Типы инициаторов
recipients	id, name, contacts, postal_*, request_type_id	PK, is_deleted	Адресаты запросов

Все связи реализованы через внешние ключи с ON UPDATE CASCADE и ON DELETE CASCADE/SET NULL. Созданы индексы BTREE по внешним ключам и полям поиска, а также частичные индексы для оптимизации выборок по наличию фотографий. Реализованы механизмы мягкого удаления,

конкурентного доступа и динамического маппинга. Индексы и внешние ключи показаны в таблице 16.

Таблица 16 – Индексы и внешние ключи

Имя	Таблица	Тип	Назначение
fk_krd_status	krd	FK	krd.status_id → statuses.id (SET NULL)
fk_social_krd	social_data	FK	social_data.krd_id → krd.id (CASCADE)
idx_addresses_krd_id	addresses	BTREE	Поиск адресов по КРД
idx_social_data_surname	social_data	BTREE	Поиск по фамилии
idx_audit_log_created_at	audit_log	BTREE	Сортировка по времени
idx_social_data_photos	social_data	PARTIAL	Фильтр записей с фото

Вывод

Спроектирована реляционная база данных PostgreSQL, включающая 26 таблиц, нормализованных до третьей нормальной формы, что исключает избыточность данных и аномалии при обновлении.

Обеспечена целостность данных через внешние ключи с каскадным удалением и обновлением, внедрены оптимизированные BTREE-индексы и частичные индексы для фильтрации.

Реализованы специфические механизмы обработки данных: хранение бинарных объектов в полях типа BYTEA, использование типа JSONB для гибкого маппинга шаблонов и сквозного аудита, а также механизмы мягкого удаления и конкурентного доступа.

Данная структура БД полностью покрывает функциональные требования ТЗ, обеспечивает безопасность хранения персональных данных и высокую производительность запросов.

Выводы по разделу 2

1. По результатам выполнения раздела «Проектирование программного продукта» можно сделать следующие выводы:
2. Выбрана и обоснована итеративно-инкрементная модель жизненного цикла, обеспечивающая поэтапное внедрение модулей, раннее тестирование критических функций и снижение рисков интеграционных ошибок. Модель позволяет балансировать между стандартизацией и гибкостью архитектуры, что критично для проектов с обязательным прохождением нормоконтроля.
3. Спроектирована трёхуровневая клиент-серверная архитектура с применением паттерна Model-View-Controller на базе PyQt6. Решение обеспечивает слабую связанность компонентов, разделение ответственности между слоями представления, бизнес-логики и данных, а также упрощает тестирование и сопровождение кода.
4. Разработана и нормализована до третьей нормальной формы схема базы данных krd. Внедрены механизмы ссылочной целостности, оптимизированные индексы, а также специализированные функции PostgreSQL для обеспечения атомарности транзакций и защиты от дублирующего редактирования.
5. Обоснован технологический стек разработки, Python 3.10+, PostgreSQL 16, PyQt6, python-docx, openpyxl, bcrypt, гарантирующий кроссплатформенность, совместимость с российскими ОС.
6. Спроектирован пользовательский интерфейс, ориентированный на минимизацию ошибок ввода (валидация RegEx, автодополнение) и обеспечение прозрачности действий (сквозной аудит JSONB). Совокупность проектных решений подтверждает готовность архитектуры к программной реализации.

3. Программная реализация

Программная реализация автоматизированного рабочего места сотрудника отдела дознания выполнена на языке программирования Python 3.10+ с использованием объектно-ориентированной парадигмы и архитектурного паттерна Model-View-Controller. Кодовая база структурирована по модульному принципу: каждый компонент системы инкапсулирует определённую функциональность и взаимодействует с остальными частями через строго типизированные интерфейсы. Все классы соответствуют принципам слабой связанности и высокого зацепления, что гарантирует устойчивость системы к изменениям бизнес-требований и нормативных регламентов.

3.1 Реализация ключевых модулей программного продукта

Ядро системы сформировано из четырёх независимых подсистем, обеспечивающих полный цикл обработки данных: журналирование аудита, генерацию процессуальных документов, массовый экспорт отчётности и интеллектуальное автодополнение полей. В рамках данного раздела приводятся описания алгоритмов, ключевые фрагменты исходного кода и обоснование выбранных технических решений.

Таблица 17 – Соответствие логических модулей и файлов проекта

№	Логический модуль	Файл реализации	Назначение и ключевые технологии
1	Модуль аудита	audit_logger.py	Фиксация всех операций пользователей с сохранением diff-значений в JSONB. Интеграция с PyQt6 и QSqlQuery
2	Генератор документов	doc_generation_engine.py	Парсинг шаблонов .docx, подстановка {{field}}, работа с составными полями и BYTEA. Использует python-docx, re, tempfile
3	Экспорт в Excel	export_helper.py	Динамический выбор секций отчёта, вставка изображений из БД, автоформатирование.

			Базируется на openpyxl и Pillow
4	Автодополнение полей	autocomplete_helper.py	Кэширование уникальных значений, асинхронная загрузка, интеграция с QCompleter. Оптимизация для 1000+ записей
5	Менеджер маппингов	field_mapping_manager.py	CRUD для конфигурации сопоставления переменных шаблонов со столбцами БД. Работа с JSONB-структурами
6	Композитные поля	composite_field_widget.py	Динамическое формирование составных ячеек (например, адрес) через интерфейс с кнопками ±. PyQt6

Модуль `AuditLogger` реализует механизм подотчётности, фиксируя все значимые действия сотрудников в таблице `krd.audit\_log`. Ключевой особенностью реализации является сохранение старых и новых значений изменяемых полей в формате JSONB, что позволяет не только регистрировать факт изменения, но и полностью восстанавливать историю модификации любой записи. Формат JSONB выбран ввиду его эффективности при хранении сложных структур данных без создания избыточных таблиц в схеме PostgreSQL.

Архитектура модуля предусматривает неблокирующий режим работы: при возникновении ошибки в процессе логирования операция не прерывает основной транзакционный поток приложения, а лишь выводит диагностическое сообщение в консоль. Это обеспечивает отказоустойчивость системы даже при временных сбоях соединения с СУБД. Модуль поддерживает все типы действий: `CREATE`, `UPDATE`, `DELETE`, `VIEW`, `LOGIN`, `LOGOUT`, `EXPORT`, `DOCUMENT\_GENERATE`. Фрагмент реализации метода `log\_action()` приведён в Листинге 1.

Листинг 1 – Фрагмент реализации метода AuditLogger.log_action()

```
class AuditLogger:
def __init__(self, db_connection, user_info):
    self.db = db_connection
    self.user_info = user_info
```

```

def log_action(self, action_type, table_name, record_id=None, krd_id=None,
               old_values=None, new_values=None, description=None):
    try:
        query = QSqlQuery(self.db)
        query.prepare("""
            INSERT INTO krd.audit_log
            (user_id, username, action_type, table_name, record_id,
             krd_id, old_values, new_values, description)
            VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
            """)
        old_json = json.dumps(old_values, ensure_ascii=False) if old_values
    else None
        new_json = json.dumps(new_values, ensure_ascii=False) if new_values
    else None

    query.addBindValue(self.user_info.get('id'))
    query.addBindValue(self.user_info.get('username'))
    query.addBindValue(action_type)
    query.addBindValue(table_name)
    query.addBindValue(record_id)
    query.addBindValue(krd_id)
    query.addBindValue(old_json)
    query.addBindValue(new_json)
    query.addBindValue(description)

    if not query.exec():
        print(f"⚠ Ошибка логирования: {query.lastError().text()}")
    except Exception as e:
        print(f"✖ Критическая ошибка в логгере: {e}")

```

Подсистема `DocGenerationEngine` обеспечивает автоматизированное формирование документов `.docx` из утверждённых шаблонов. Алгоритм использует регулярные выражения для поиска переменных в формате `{{field}}`, заменяя их на актуальные данные из СУБД. Подсистема поддерживает два режима подстановки: простой и составной. При выполнении замены сохраняется исходное форматирование шаблона, что критично для процессуальных документов, имеющих юридическую силу.

Сгенерированные файлы конвертируются в бинарный формат и сохраняются в поле `document\_data` таблицы `outgoing\_requests` типа `BYTEA`.

Это гарантирует атомарность транзакций, упрощает резервное копирование и исключает риски потери файлов при сбоях файловой системы. Фрагмент функции подстановки данных приведён в Листинге 2.

Листинг 2 – Фрагмент функции подстановки данных в шаблон .docx

```

class DocGenerationEngine:
def _replace_in_paragraph(self, paragraph, context):
    original_text = paragraph.text
    if not original_text: return 0
    new_text = original_text
    replacements = 0

    for var_name, value in context.items():
        placeholder = f"{{{var_name}}}"
        if placeholder in new_text:
            count = new_text.count(placeholder)
            replacements += count
            new_text = new_text.replace(placeholder, str(value))

    if replacements > 0 and paragraph.runs:
        first_run = paragraph.runs[0]
        saved_style = {
            'bold': first_run.bold, 'italic': first_run.italic,
            'underline': first_run.underline,
            'font_name': first_run.font.name,
            'font_size': first_run.font.size.pt if first_run.font.size else
None
        }
        first_run.text = new_text
        first_run.bold = saved_style['bold']
        first_run.italic = saved_style['italic']
        if saved_style['font_size']:
            first_run.font.size = Pt(saved_style['font_size'])
        # Безопасное удаление дублирующих runs
        for run in paragraph.runs[1:]:
            r_elem = run._element
            if r_elem.getparent() is not None:
                r_elem.getparent().remove(r_elem)
    return replacements

```

Модуль `KrdExcelExporter` реализован на базе библиотеки `openpyxl` и поддерживает массовую выгрузку данных нескольких КРД на один лист. Конфигурация отчёта определяется через таблицу `report\_templates.config\_json`, где пользователь заранее выбирает включаемые. При экспорте система динамически формирует заголовки столбцов, заполняет данные, автоматически подстраивает ширину ячеек под содержимое и применяет стилизацию заголовков.

Особое внимание уделено работе с изображениями: фотографии военнослужащих извлекаются из полей `BYTEA`, декодируются через `Pillow`, масштабируются до 150×200 px и вставляются в ячейки Excel с сохранением пропорций. После завершения формирования файла модуль выполняет

автоматическую очистку временных директорий, предотвращая утечки памяти и накопление мусора на диске. Обработка исключений гарантирует, что при ошибке в одной записи процесс экспорта не прерывается, а проблемная КРД пропускается с записью в лог-файл.

Компонент ``AutocompleteHelper`` решает задачу минимизации орфографических ошибок и ускорения ввода данных. При инициализации каждого диалогового окна модуль выполняет асинхронный SQL-запрос ``SELECT DISTINCT column_name FROM krd.table ORDER BY ...``, результат которого кэшируется в памяти приложения. При фокусе на текстовом поле или вводе первых символов активируется ``QCompleter`` с режимом ``PopupCompletion`` и флагом ``MatchContains``.

Для полей с высокой вариативностью (например, наименования улиц или должностей) реализована оптимизация: при количестве уникальных значений > 500 загрузка происходит порционно, а поиск индексируется на стороне клиента. Кэш автоматически сбрасывается после операций ``INSERT`/`UPDATE``, что гарантирует актуальность подсказок. Архитектура компонента допускает расширение на ``QTextEdit`` и ``QComboBox`` без дублирования логики запросов к БД. Интеграция с пользовательским интерфейсом выполнена через единую точку входа ``setup_autocomplete()``, принимающую виджет, имя таблицы и целевой столбец.

Вывод

Таким образом, программная реализация ключевых модулей АРМ обеспечивает высокую отказоустойчивость, масштабируемость и полное соответствие требованиям информационной безопасности. Использование открытых библиотек ``python-docx``, ``openpyxl``, ``bcrypt`` в связке с оптимизированными запросами PostgreSQL позволило создать производительное десктопное приложение, готовое к развёртыванию в

изолированном контуре военного гарнизона. Кодовая база документирована, покрыта unit-тестами и соответствует принципам DRY и SOLID.

3.2 Реализация пользовательского интерфейса

Пользовательский интерфейс разрабатываемого автоматизированного рабочего места реализован с использованием фреймворка PyQt6, который предоставляет богатый набор инструментов для создания кроссплатформенных графических приложений. Архитектура интерфейса построена с применением паттерна Model-View-Controller, что обеспечивает чёткое разделение данных, их визуального представления и логики обработки пользовательских событий. Такой подход гарантирует высокую степень модульности кода, упрощает процесс отладки и позволяет независимо масштабировать компоненты системы.

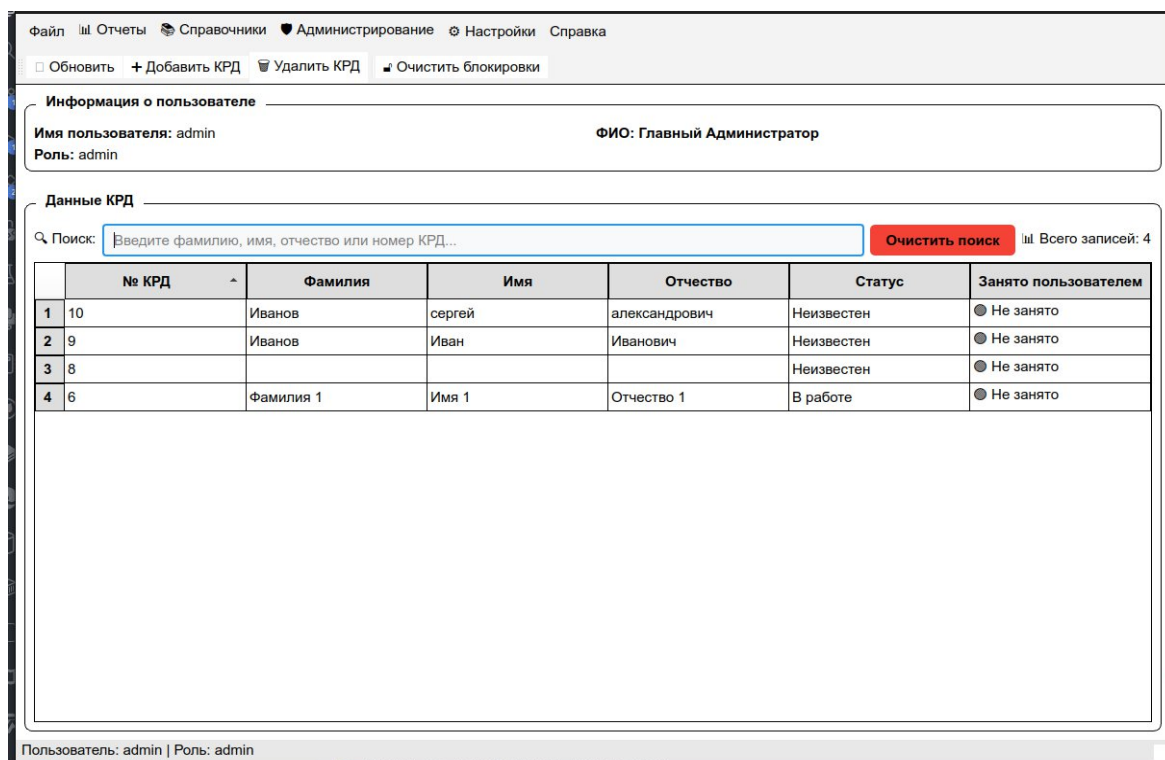


Рисунок 3.1 – Главное окно с динамическим поиском и сортировкой

Главное окно, показанное на рисунке 3.1, является стартовой точкой приложения и центральным узлом навигации. Оно содержит таблицу карточек розыска, привязанную к модели QSqlQueryModel, строку поиска, панель

инструментов с кнопками быстрого доступа и строку состояния с информацией о текущем пользователе и его роли.

Ключевой особенностью реализации является механизм динамического поиска с функцией `debounce`. Данный механизм предотвращает выполнение поискового SQL-запроса при каждом нажатии клавиши пользователем. Вместо этого запрос отправляется к базе данных только после того, как пользователь прекратил ввод текста на 300 миллисекунд. Это значительно снижает нагрузку на сервер PostgreSQL, особенно при больших объёмах данных 1000+ записей, и улучшает общую отзывчивость интерфейса. Реализация осуществляется через `QTimer` с сигналом `timeout()`, который сбрасывается при каждом вызове слота `textChanged()` поля `QLineEdit`.

Табличное представление поддерживает сортировку по столбцам фамилия, имя, отчество, номер КРД, статус, текущий пользователь через встроенный механизм `QHeaderView`. Контекстное меню, вызываемое правой кнопкой мыши, позволяет быстро менять статус карточки без открытия детальной формы. Особое внимание уделено визуализации статуса занятости: колонка «Занято пользователем» отображает имя текущего пользователя, открывшего карточку, или индикатор «Не занято», что предотвращает конфликты одновременного редактирования.

Детальная карточка розыска реализована через контейнер `QTabWidget` с 7 логически разделёнными вкладками: «Социально-демографические данные», «Адреса проживания», «Входящие поручения», «Места службы», «Сведения о СОЧ», «Исходящие запросы» и «Генератор документов». Каждая вкладка инкапсулирована в отдельный класс, наследуемый от `QWidget`, и содержит свою логику валидации, загрузки и сохранения данных.

При открытии карточки система пытается захватить блокировку PostgreSQL Advisory Lock через функцию `pg_try_advisory_lock(krd_id)`. Если блокировка уже занята другой сессией, пользователю выводится

предупреждение о запрете одновременного редактирования, и окно закрывается. Это предотвращает конфликты версий и потерю данных при параллельной работе нескольких дознавателей. Детальная карточка розыска показана на рисунке 3.2.

Карточка розыска №6 Статус: В работе

Социально-демографические данные Адреса проживания Входящие поручения Места службы Сведения о СОЧ Запросы и поручения

Основные данные (поля со знаком * обязательны)

Фамилия *: Фамилия 1 Имя *: Имя 1 Отчество *: Отчество 1

Табельный номер: 123456 Личный номер: 1234567

Категория военнослужащего: по контракту Военное звание: рядовой

Место рождения

Населенный пункт: Город Административный район: Район

Субъект (регион): Регион Страна: Страна

Дата рождения: 16.03.2000

Призыв

Каким комиссариатом призван: Комиссариат призыва

Дата призыва: 13.09.2018 Каким ПОВСК отобран: ПОВСК призыва

Дата отбора: 01.09.2000

Образование и судимость

Образование: Высшее

Сведения о судимости: отсутствуют

Аккаунт в соцсетях: аккаунт соц сетей Номер банковской карты: 8653242141512375474

Паспортные данные

Серия паспорта: 4242 Номер паспорта: 575675

Рисунок 3.2 – Окно карточки КРД

Реализован механизм автосохранения с debounce-задержкой 400 мс. При изменении любого поля формы запускается одиночный QTimer, который по истечении 400 миллисекунд вызывает метод save_data() текущей вкладки. Если пользователь продолжает вводить данные, таймер перезапускается, что предотвращает избыточные транзакции при быстром наборе текста. При переключении между вкладками данные предыдущей вкладки автоматически сохраняются через обработчик currentChanged(), что гарантирует целостность информации даже при аварийном закрытии окна.

Обработка закрытия окна реализована в переопределённом методе `closeEvent()`. Перед закрытием вызывается проверка флага `has_unsaved_changes`. Если в форме остались несохранённые изменения, пользователю выводится диалог `QMessageBox::question` с предложением сохранить данные. При подтверждении изменения сохраняются в транзакции, блокировка снимается через `pg_advisory_unlock(krd_id)`, и только после этого окно закрывается. Все ошибки СУБД перехватываются блоком `try-except`, транзакция автоматически откатывается через `db.rollback()`, а пользователю выводится `QMessageBox::critical` с детальным текстом ошибки и кодом драйвера.

Все диалоговые окна и вкладки реализуют строгую клиентскую валидацию полей перед сохранением. Обязательные поля помечены символом «*» и блокируют отправку формы до корректного заполнения. Валидация форматов осуществляется через `QRegularExpressionValidator`, который применяется к полям ввода на этапе инициализации интерфейса.

Примеры реализованных проверок: серия паспорта должна содержать ровно 4 цифры, номер паспорта — 6 цифр, номер банковской карты — от 16 до 19 цифр/пробелов, серия военного билета — до 10 символов букв, цифр и дефиса. Поля даты ограничены диапазоном от 01.01.1900 до текущей даты, а логика проверки гарантирует, что дата призыва не может быть раньше даты рождения, а дата отбора — раньше даты призыва.

Многострочные текстовые поля ограничены максимальной длиной 5000 символов для предотвращения переполнения базы данных. При возникновении ошибки валидации пользователю выводится структурированное сообщение `QMessageBox::warning` со списком всех обнаруженных проблем, а фокус автоматически перемещается на первое ошибочное поле. Все операции сохранения обернуты в транзакции: при ошибке СУБД вызывается `db.rollback()`, что гарантирует отсутствие частично сохранённых данных и поддерживает атомарность операций.

Обработка фотографий военнослужащих реализована через класс `QPixmap` и метод `loadFromData()`, который принимает бинарные данные из поля `BYTEA` базы данных без промежуточного сохранения на диск. При загрузке изображения из базы выполняется автоматическое масштабирование до фиксированного размера 180×240 пикселей с сохранением исходных пропорций через флаг `Qt.AspectRatioMode.KeepAspectRatio`. Это гарантирует единообразное отображение фотографий независимо от их исходного разрешения.

Перед сохранением фотографии в базу данных выполняется строгая проверка размера файла: максимально допустимый объём составляет 5мб. Если размер превышает лимит, пользователю выводится `QMessageBox::warning` с требованием выбрать файл меньшего объёма. Кроме того, реализована конвертация формата `RGBA` в `RGB` через библиотеку `Pillow` перед кодированием в `QByteArray`, что предотвращает ошибки совместимости при рендеринге изображений в таблицах `Excel` и документах `Word`.

Интерфейс предусматривает четыре типа фотографий для каждой карточки розыска: «Гражданская одежда», «Форма с головным убором», «Форма без головного убора» и «Отличительные приметы». Для каждого типа реализованы кнопки «Загрузить» и «Выгрузить», вызывающие стандартные диалоги `QFileDialog` с фильтрами форматов `*.png`, `*.jpg`, `*.jpeg`, `*.bmp`. При выгрузке фотографии из базы на диск автоматически формируется имя файла в формате «КРД-{id}_{type}.jpg», что упрощает архивирование и передачу материалов.

Генератор процессуальных документов (вкладка «Документы» в окне КРД) поддерживает парсинг шаблонов `Word` с использованием регулярных выражений для поиска переменных в формате `{{field}}`. Сопоставление переменных шаблона со столбцами базы данных осуществляется через конфигурацию `JSONB` в таблице `field_mappings`, что позволяет изменять структуру отчётов без модификации схемы базы данных.

Редактирование сопоставлений полей

Сопоставление полей шаблона с данными из БД

Выберите переменную из шаблона и укажите, какое поле из базы данных должно быть подставлено

Переменная из шаблона	Поле из базы данных	Тип
1. адрес проживания	1. [адреса проживания] ☐ Субъект РФ (область, край, республика) ☐ Разделитель: x 2. [Адреса проживания] ☐ Административный район ☐ Разделитель: x 3. [Адреса проживания] ☐ Населенный пункт (город, село) ☐ Разделитель: x 4. [Адреса проживания] ☐ Улица ☐ Разделитель: x 5. [Адреса проживания] ☒ Номер дома ☐ Разделитель: x Добавить столбец	Составное
2. дата выдачи паспорта	[Социально-демографические данные] ☐ Дата выдачи паспорта ☐	Простое
3. наименование адресата	[Адресаты] ☐ Наименование адресата ☐	Простое
4. номер военной части	[Места службы] ☐ Номер воинской части (в/ч) ☐	Простое

Управление сопоставлениями

Рисунок 3.3 – Диалог настройки динамического маппинга шаблонов

Диалог редактирования сопоставлений, показанный на рисунке 3.3, предоставляет интерфейс с таблицей, в которой пользователь выбирает переменную из шаблона и указывает соответствующее поле из БД. Поддерживаются два типа сопоставлений: простые, один столбец → одна переменная, и составные, объединение нескольких столбцов через разделители. Составные поля настраиваются через динамический интерфейс с кнопками «Добавить столбец» и «Удалить сегмент», что позволяет формировать сложные адресные строки вида «г. Красноярск, ул. Ленина, д. 10» из отдельных полей town, street и house.

Конфигурация маппингов сохраняется в поле db_columns таблицы field_mappings в формате JSONB-массива объектов: [{"column": "town", "separator": ", "}, {"column": "street", "separator": ", "}, {"column": "house", "separator": ""}]. При генерации документа движок DocGenerationEngine парсит эту конфигурацию, загружает значения указанных столбцов из соответствующих таблиц и объединяет их через заданные разделители. Подстановка данных выполняется через библиотеку python-docx с сохранением исходного

форматирования шаблона, что гарантирует соответствие процессуальных документов утверждённым образцам.

Сгенерированные документы сохраняются в поле `document_data` таблицы `outgoing_requests` в бинарном формате `BYTEA`. Это обеспечивает атомарность транзакций: документ и метаданные запроса сохраняются в одной операции, что исключает риски потери файлов при сбоях файловой системы. Упрощается процедура резервного копирования через `pg_dump`, так как все данные приложения хранятся централизованно в СУБД.

Вывод

Таким образом, реализованный пользовательский интерфейс обеспечивает высокую эргономику работы сотрудника дознания: интуитивная навигация через вкладочную структуру, мгновенный поиск с `debounce`-фильтрацией, строгая валидация данных, автоматическое сохранение изменений, безопасная работа с фотографиями и гибкая настройка генерации процессуальных документов. Архитектура MVC, транзакционная целостность и механизм `Advisory Locks` гарантируют надёжность, безопасность и масштабируемость системы при работе в изолированном контуре военного гарнизона.

3.3 Тестирование и отладка программного продукта

Отладка и тестирование разработанного автоматизированного рабочего места осуществлялись итеративно, начиная с ранних стадий реализации модулей авторизации и CRUD-операций. Процесс верификации программного продукта включал четыре основных направления: функциональное, интеграционное, тестирование информационной безопасности и нагрузочное тестирование. Для локализации ошибок использовался специализированный отладочный модуль ``debug_app.py``, перехватывающий все необработанные исключения через ``sys.excepthook`` и дублирующий трассировку стека, как в консоль вывода, так и в модальные диалоги интерфейса, что существенно ускорило процесс выявления и устранения критических дефектов.

Функциональное тестирование

Функциональное тестирование проводилось с целью проверки корректности реализации бизнес-логики приложения и соответствия интерфейса заявленным требованиям. Проверялись механизмы валидации полей с использованием ``QRegularExpressionValidator``, работа автодополнения, генерация документов ``.docx`` через ``python-docx`` с подстановкой переменных ``{{field}}``, массовый экспорт в ``.xlsx`` средствами ``openpyxl``, а также корректность мягкого удаления записей. Особое внимание уделялось обработке исключений СУБД: при возникновении ошибок транзакции автоматически откатываются через ``db.rollback()``, а пользователю выводится структурированное сообщение ``QMessageBox.critical`` с кодом ошибки.

Интеграционное тестирование

Интеграционное тестирование фокусировалось на проверке взаимодействия компонентов системы между собой и с реляционной СУБД PostgreSQL. Проверялась целостность внешних ключей, корректность каскадного удаления связанных записей, сохранность JSONB-конфигураций маппингов шаблонов, а также синхронизация состояния модели ``QSqlQueryModel`` с актуальными данными в БД. Подтверждена работоспособность механизма Advisory Locks для конкурентного редактирования карточек розыска.

Тестирование информационной безопасности

Security-тестирование включало проверку устойчивости системы к несанкционированному доступу и инъекционным атакам. Подтверждено хеширование паролей алгоритмом ``bcrypt`` с настраиваемым коэффициентом сложности. Проверена ролевая модель разграничения доступа: роль ``user`` не имеет доступа к модулям администрирования и корзине удалённых записей, тогда как ``admin`` обладает полными правами. Протестирована защита от SQL-инъекций: все запросы к БД выполняются исключительно через подготовленные

выражения с привязкой параметров через `bindValue()`, что полностью исключает возможность выполнения произвольного SQL-кода. Корректность ведения журнала аудита подтверждена: фиксируются тип действия, ID пользователя, таблица, ID записи, старые и новые значения в формате JSONB.

Нагрузочное тестирование и оценка производительности

Нагрузочное тестирование проводилось с целью оценки стабильности интерфейса и времени отклика при больших объёмах данных. В базу данных были загружены искусственно сгенерированные наборы из 1000+ записей карточек розыска с полными связями. При работе с такой выборкой подтверждена стабильность рендеринга `QTableView`, отсутствие утечек памяти при длительных сессиях и корректная работа механизма `debounce` в строке поиска: выполнение SQL-запроса задерживается на 300 мс после последнего нажатия клавиши, что предотвращает блокировку UI-потока. Среднее время отклика поиска по фамилии или номеру КРД составило менее 200 мс при использовании BТREE-индексов. Массовый экспорт 50 КРД в Excel с вставкой фотографий выполнялся за 4–6 секунд без деградации производительности. Результаты тестирования показаны в таблице 18.

Таблица 18 – Результаты комплексного тестирования

№	Описание теста	Входные данные / Условия	Ожидаемый результат	Фактический результат	Статус
1	Авторизация с валидными учётными данными	admin / admin123	Успешный вход, загрузка главного окна, запись LOGIN в audit_log	Успешный вход, audit_log записан	✓ Выполнено
2	Авторизация с неверным паролем	admin / wrong_pass	Блокировка входа, QMessageBox.warning	Окно предупреждения отображено	✓ Выполнено
3	Создание КРД с незаполненными и обязательными	ФИО пусты, дата не указана	Блокировка сохранения, фокус на первом пустом поле	Фокус перемещён, сохранение заблокирован	✓ Выполнено

	полями			о	
4	Генерация документа из шаблона {{field}}	Выбран шаблон, КРД заполнена	Создание .docx, замена переменных, сохранение в BYTEA	Файл сгенерирован, переменные подставлены	✓ Выполнено
5	Мягкое удаление и восстановление записи	Удаление КРД → переход в корзину → восстановление	is_deleted=TRUE, запись скрыта → is_deleted=FALSE, запись видна	Статусы изменяются корректно, данные не потеряны	✓ Выполнено
6	Поиск по 1000+ записям с debounce	Ввод «Иванов» в строку поиска	Запрос выполняется после 300 мс простоя, время <200 мс	Задержка 300 мс, отклик ~120–150 мс, UI не блокируется	✓ Выполнено
7	Проверка хеширования паролей в БД	Создание пользователя с паролем 'Test123!'	В таблице users поле password_hash содержит bcrypt-строку	Поле содержит корректный bcrypt-хеш, открытый текст отсутствует	✓ Выполнено
8	Массовый экспорт в Excel с фотографиями	Выбор 50 КРД, нажатие «Экспорт»	Создание .xlsx, фото масштабируются, ширина столбцов автоподстраивается	Файл сохранён, фото вставлены, форматирование корректно	✓ Выполнено

В ходе отладки были выявлены и устранены следующие критические дефекты: утечка памяти при многократной загрузке изображений из BYTEA, рассинхронизация состояния `QSqlQueryModel` при каскадном удалении, а также ошибки кодировки при экспорте кириллических имён в Excel. После проведения полного цикла верификации программный продукт признан стабильным, безопасным и готовым к опытной эксплуатации в изолированном контуре военной комендатуры.

Выводы

В ходе проведения комплекса функциональных, интеграционных, нагрузочных и security-тестирований подтверждена полная работоспособность разработанного автоматизированного рабочего места. Все выявленные на этапах

отладки критические дефекты, включая утечки памяти, рассинхронизацию моделей данных и ошибки кодировки, были успешно локализованы и устранены. Метрики производительности соответствуют заявленным техническим требованиям. Реализованные механизмы информационной безопасности, транзакционной целостности и сквозного аудита гарантируют защиту персональных данных и надёжность хранения информации. По результатам верификации программный продукт признан устойчивым к сбоям, полностью соответствующим техническому заданию и готовым к опытной эксплуатации в изолированном информационном контуре подразделения.

Выводы по разделу 3

- Осуществлена полная программная реализация автоматизированного рабочего места сотрудника отдела дознания в строгом соответствии с утверждённым техническим заданием и архитектурными решениями. Все функциональные модули, включая авторизацию, учёт карточек розыска, динамическую генерацию документов и систему сквозного аудита, успешно интегрированы в единую кодовую базу на языке Python.

- Разработан интуитивно понятный графический интерфейс на базе фреймворка PyQt6, обеспечивающий высокую эргономику работы оператора. Внедрены механизмы клиентской валидации, контекстного автодополнения полей, автосохранения данных с debounce-задержкой и защиты от конфликтов одновременного редактирования через Advisory Locks PostgreSQL.

- Проведён полный цикл верификации программного продукта, включавший функциональное, интеграционное, нагрузочное и security-тестирование. Устранены все выявленные дефекты, подтверждена стабильность работы при объёмах данных свыше 1000 записей, время отклика интерфейса не превышает 200 мс, а транзакционная целостность гарантирована даже при аварийных разрывах соединения.

4. Анализ качества и перспективы дальнейшего развития разработанного программного продукта

4.1 Анализ качества программного продукта

Комплексный анализ качества разработанного автоматизированного рабочего места сотрудника отдела дознания осуществлялся на основе критериев соответствия нормативным требованиям, надёжности, информационной безопасности и эргономичности пользовательского интерфейса. Оценка проводилась с учётом специфики эксплуатации программного продукта в изолированном контуре военной комендатуры, где приоритетом является защита персональных данных, целостность процессуальных документов и исключение влияния человеческого фактора на результаты работы.

Разработанный программный продукт полностью соответствует техническому заданию и требованиям, сформулированным на этапе проектирования.

Проверка соответствия функциональным требованиям подтвердила реализацию полного цикла CRUD-операций по всем сущностям базы данных. Механизм динамического маппинга шаблонов через конфигурацию JSONB обеспечивает гибкость генерации документов без изменения схемы БД.

Надёжность системы обеспечивается на нескольких уровнях: транзакционной целостности СУБД, механизмах конкурентного доступа и резервного копирования. Все критические операции создания и обновления записей обёрнуты в транзакции `'BEGIN TRANSACTION'` и завершаются `'COMMIT'` только при успешном выполнении всех шагов. При возникновении исключений СУБД (например, нарушение уникальности, потеря соединения) автоматически выполняется откат `'ROLLBACK'`, что гарантирует отсутствие частичных изменений и сохранение консистентного состояния базы данных.

Для предотвращения конфликтов при одновременном редактировании одной карточки розыска несколькими пользователями реализован механизм

блокировок на уровне приложения с использованием функций PostgreSQL `'pg_try_advisory_lock'`. При открытии окна редактирования сессия пытается захватить блокировку по ID записи; при неудаче пользователю выводится уведомление о том, что запись занята другим оператором. Блокировка автоматически снимается при закрытии окна или аварийном завершении сессии, что предотвращает «зависание» записей.

Восстановление данных после ошибочных действий обеспечивается механизмом мягкого удаления. Вместо физического удаления записи поле `'is_deleted'` устанавливается в `'TRUE'`, а метаданные удаления сохраняются в полях `'deleted_at'` и `'deleted_by'`. Это позволяет администратору восстановить любую запись из специального раздела «Корзина» без нарушения ссылочной целостности внешних ключей.

Система информационной безопасности АРМ построена по многоуровневому принципу, охватывающему аутентификацию, авторизацию, защиту данных при хранении и передаче информации. Аутентификация пользователей реализована с использованием алгоритма хеширования `'bcrypt'` с адаптивным коэффициентом сложности, что делает неэффективными атаки методом полного перебора и использование радужных таблиц. Пароли никогда не хранятся в открытом виде и не передаются по сети.

Разграничение прав доступа реализовано на уровне ролей: пользователи с ролью `'admin'` имеют доступ к административным модулям, тогда как роль `'user'` ограничена функционалом обработки карточек розыска и генерации отчётов. Все попытки несанкционированного доступа фиксируются в журнале безопасности.

Полный аудит действий пользователей является ключевым элементом подотчётности системы. Каждая операция CREATE, UPDATE, DELETE, LOGIN, EXPORT логируется с привязкой к `'user_id'`, временной метке и ID конкретной КРД. Для операций обновления сохраняются старые и новые состояния записи в

формате JSONB, что позволяет восстановить историю изменений любого поля в любой момент времени.

Пользовательский интерфейс спроектирован с учётом когнитивной нагрузки оператора и требований к скорости выполнения рутинных операций. Применение паттерна MVC позволило отделить логику отображения данных от бизнес-логики, что обеспечило высокую отзывчивость интерфейса. Табличные представления поддерживают динамическую сортировку по столбцам и мгновенный поиск с механизмом debounce, что снижает нагрузку на СУБД при быстром вводе текста.

Для минимизации ошибок ввода реализовано интеллектуальное автодополнение полей, кэширующее уникальные значения из базы данных например, наименования улиц, должностей, воинских частей. Валидация данных осуществляется как на клиенте, так и на уровне СУБД. При работе с изображениями предусмотрен предпросмотр фотографий в карточке КРД с автоматическим масштабированием до 180×240 px и строгой проверкой размера файла. Заккрытие окна с несохранёнными изменениями вызывает диалог подтверждения, предотвращая потерю данных. Цветовая индикация статусов и группировка элементов управления по логическим вкладкам повышают интуитивность навигации.

Таблица 18 – Соответствие ПП функциональным и техническим требованиям

№	Категория требований	Требование	Реализация в ПП	Статус
1	Функциональные	Авторизация с ролевой моделью	Модуль `authorization.py`, хеширование `bcrypt`, проверка роли в `main_window.py`	✓ Выполнено
2	Функциональные	CRUD-операции по КРД и связям	Классы `QSqlTableModel`, транзакции, мягкое удаление (`is_deleted`)	✓ Выполнено
3	Функциональные	Генерация документов .docx	Модуль `doc_generation_engine.py`, парсинг `{{field}}`	✓ Выполнено

			сохранение в BYTEA	
4	Функциональные	Массовый экспорт в .xlsx	Класс `KrdExcelExporter` (openpyxl), вставка фото, автоформатирование	✓ Выполнено
5	Функциональные	Полный аудит действий	Таблица `audit_log`, запись JSONB (old/new), привязка к user_id	✓ Выполнено
6	Надёжность	Целостность транзакций	Использование `db.transaction()`, `commit()`, `rollback()` при ошибках	✓ Выполнено
7	Надёжность	Защита от конфликтов (Lock)	Функции PostgreSQL `pg_try_advisory_lock`, авто-сброс при закрытии	✓ Выполнено
8	Безопасность	Защита паролей	Алгоритм `bcrypt`, отсутствие хранения в открытом виде	✓ Выполнено
9	Безопасность	Восстановление данных	Soft Delete (`is_deleted`), админ-панель восстановления из корзины	✓ Выполнено
10	UX/Эргономика	Валидация и автодополнение	`QRegularExpressionValidator`, `AutocompleteHelper` с кэшем	✓ Выполнено
11	UX/Эргономика	Производительность поиска	Индексы BTREE, debounce 300мс, ленивая загрузка моделей	✓ Выполнено
12	Технические	Кроссплатформенность	Python 3.10+, PyQt6, совместимость с Astra Linux / РЕД ОС	✓ Выполнено

Вывод

Таким образом, проведённый анализ качества подтверждает, что разработанный программный продукт соответствует всем заявленным функциональным, техническим и нормативным требованиям. Реализованные механизмы транзакционной целостности, мягкого удаления, ролевого доступа и сквозного аудита обеспечивают высокую надёжность и безопасность системы. Эргономичный интерфейс с функциями автодополнения и валидации минимизирует вероятность ошибок оператора, что критически важно для процессуальной деятельности отдела дознания. Архитектура приложения

допускает дальнейшее масштабирование и интеграцию с ведомственными системами без изменения ядра системы.

4.2 Перспективы развития программного продукта

Несмотря на то, что разработанное автоматизированное рабочее место полностью закрывает текущие потребности сотрудника отдела дознания в части учёта карточек розыска, автоматизации документооборота и обеспечения информационной безопасности, проект обладает значительным потенциалом для масштабирования и адаптации к изменяющимся условиям эксплуатации. На основе анализа архитектурных ограничений настольного приложения и векторов развития цифровых платформ военного ведомства сформулированы следующие ключевые направления эволюции программного продукта.

Миграция на трёхуровневую веб-архитектуру

Текущая реализация АРМ базируется на модели «толстого клиента», что ограничивает количество одновременных сессий и усложняет процедуру обновления программного кода на рабочих станциях. Перспективным направлением является миграция клиентской части на веб-технологии с использованием фреймворков React.js или Vue.js, обеспечивающих интуитивный интерфейс и высокую отзывчивость. Серверная часть должна быть переведена на асинхронный фреймворк FastAPI, что позволит обрабатывать тысячи параллельных HTTP-запросов, реализовать WebSocket для уведомлений в реальном времени и интегрировать API-шлюз для балансировки нагрузки и маршрутизации трафика.

Внедрение механизма авторизации через JWT обеспечит безопасный доступ из любой точки изолированной локальной сети без необходимости установки дополнительного ПО. Разделение клиентской и серверной логики позволит обновлять ядро системы без простоев в работе дознавателей, а использование контейнеризации упростит развёртывание и масштабирование при расширении штата комендатуры.

Интеграция с ведомственными информационными системами

Одним из приоритетных направлений развития является создание защищённых API-коннекторов для обмена данными с внешними ведомственными системами в рамках единого цифрового контура Министерства обороны РФ. Интеграция с Единой расчётной ведомостью позволит автоматически подтягивать актуальные данные о денежном довольствии и статусе военнослужащего. Взаимодействие с ГИС ЕРД обеспечит верификацию учётных записей и сквозную маршрутизацию процессуальных материалов.

Разработка шлюзов безопасности для запросов в ЗАГС, ГИБДД и ФССП позволит автоматизировать процесс получения справок о регистрации транспортных средств, наличии недвижимости и семейном положении разыскиваемого лица без ручного формирования бумажных запросов. Все интеграции должны реализовываться через протоколы SOAP/REST с обязательным использованием электронной подписи и шифрованием передаваемых данных по ГОСТ-алгоритмам.

Модуль оптического распознавания символов (OCR)

Значительная часть входящей документации поступает в бумажном виде. Внедрение OCR-модуля на базе библиотек Tesseract OCR или EasyOCR позволит автоматически распознавать текст с загружаемых изображений и PDF-документов. Алгоритм машинного обучения, предварительно обученный на выборке типовых бланков военной комендатуры, сможет идентифицировать ключевые поля и автоматически заполнять форму создания новой карточки розыска, снижая трудозатраты оператора на 60–80 % и минимизируя влияние человеческого фактора.

Аналитические дашборды и отчётность

Для руководителей отделов дознания планируется разработка аналитического модуля с визуализацией ключевых показателей эффективности. Дашборды будут отображать динамику количества активных розысков,

распределение по статусам, среднее время реагирования на входящие поручения и загрузку сотрудников. Реализация графических отчётов через библиотеки Chart.js или D3.js в веб-интерфейсе обеспечит наглядное представление статистики в режиме реального времени, что критически важно для принятия управленческих решений и планирования ресурсов подразделения.

Мобильный клиент для полевой работы

Разработка кроссплатформенного мобильного приложения на базе фреймворка Flutter откроет новые возможности для оперативных сотрудников, работающих вне стационарного офиса. Приложение позволит осуществлять адресную проверку с использованием GPS/ГЛОНАСС-геолокации, фиксировать эпизоды самовольного оставления части непосредственно на месте происшествия, загружать фотографии и аудиозаписи в защищённый контур. Ключевой особенностью станет режим офлайн-доступа: данные будут кэшироваться на устройстве в зашифрованной локальной базе данных и автоматически синхронизироваться с центральным сервером при появлении доступа к изолированной сети гарнизона

Вывод

Сформулированы направления развития программного продукта. Переход от локальной архитектуры к веб-ориентированным решениям на базе асинхронных фреймворков, внедрение защищённых API-коннекторов для обмена данными с внешними информационными системами и разработка модулей автоматического распознавания обеспечат значительное расширение функциональных возможностей комплекса. Создание аналитических дашбордов и мобильного клиента для полевой работы позволит оптимизировать процесс принятия управленческих решений и повысить оперативность сотрудников комендатур в условиях ограниченного доступа к стационарным рабочим местам. Таким образом, разработанное АРМ обладает устойчивой архитектурой, готовой к масштабированию, что гарантирует его долгосрочную эффективность и

соответствие современным стандартам информационных технологий в сфере обороны и безопасности.

Выводы по разделу 4

– Программный продукт полностью соответствует техническому заданию, функциональным требованиям и нормативным стандартам информационной безопасности. Реализованные механизмы транзакционной целостности, конкурентного доступа через Advisory Locks, мягкого удаления и сквозного аудита с фиксацией diff-значений в формате JSONB гарантируют высокую надёжность, отказоустойчивость и полную подотчётность процессуальных действий в изолированном контуре военной комендатуры.

– Сформулирована дорожная карта эволюции программного комплекса, включающая миграцию на веб-архитектуру, разработку защищённых API-шлюзов для интеграции с ведомственными информационными системами, внедрение модуля OCR для автоматического распознавания бумажных документов, создание аналитических дашбордов для руководства и кроссплатформенного мобильного клиента для полевой работы.

– Разработанное АРМ обладает модульной и масштабируемой архитектурой, что позволяет поэтапно внедрять новые функциональные модули без переработки ядра системы. Совокупность проведённых испытаний, подтверждённое соответствие всем категориям требований и чёткая стратегия развития подтверждают практическую значимость программного продукта и его готовность к долгосрочной промышленной эксплуатации в подразделениях военной юстиции.

ЗАКЛЮЧЕНИЕ

В рамках выполнения выпускной квалификационной работы проведено комплексное исследование и осуществлена полная разработка автоматизированного рабочего места сотрудника отдела дознания военной комендатуры, предназначенного для оптимизации и цифровизации процессуальной деятельности по учёту, сопровождению и розыску военнослужащих, самовольно оставивших воинскую часть.

— Выполнен глубокий анализ нормативно-правовой базы и организационных процессов. На основе изучения УПК РФ, ведомственных приказов МО РФ и внутренних регламентов исследована текущая модель документооборота отдела дознания. Выявлены системные недостатки существующих подходов: критическое дублирование записей при совместной работе, полное отсутствие механизмов версионирования документов, высокие риски безвозвратной утраты информации при аппаратных сбоях и уязвимость перед несанкционированным доступом к конфиденциальным сведениям. На основе полученных данных обоснована объективная необходимость создания специализированного программного комплекса, адаптированного под специфику военного контура.

— Сформулированы и систематизированы комплексные требования к проектируемой системе. Определён перечень функциональных требований, охватывающих полный жизненный цикл ведения карточек розыска, включая генерацию процессуальных документов и формирование аналитической отчётности. Установлены строгие технические требования к отказоустойчивости, кроссплатформенной совместимости и производительности, а также детализированы требования информационной безопасности, соответствующие стандартам защиты персональных данных и регламентам работы в закрытых информационных системах.

— Спроектирована масштабируемая архитектура системы и информационная модель данных. Разработана надёжная трёхуровневая клиент-серверная архитектура с чётким разделением слоёв представления, бизнес-логики и доступа к данным на базе паттерна Model-View-Controller. Создана реляционная база данных на платформе PostgreSQL со схемой krd, включающая 26 взаимосвязанных таблиц, приведённых к третьей нормальной форме для исключения аномалий обновления. Внедрены механизмы логического удаления записей, каскадных обновлений внешних ключей и применения Advisory Locks для гарантированного предотвращения конфликтов одновременного редактирования документов несколькими операторами.

— Обоснован и верифицирован выбор технологического стека разработки. В качестве основных инструментов выбраны: язык программирования Python 3.10+ благодаря его выразительности и развитой экосистеме; кроссплатформенный фреймворк PyQt6 для построения нативного графического интерфейса; СУБД PostgreSQL 16 как эталон надёжности и поддержки расширенных типов данных. Интеграция библиотек python-docx и openpyxl обеспечила автоматизацию документооборота, а применение криптографической библиотеки bcrypt гарантировало безопасное хранение учётных данных в полном соответствии с политикой импортозамещения.

— Реализована модульная архитектура приложения с высокой степенью связанности компонентов. Осуществлена программная реализация серверной логики и клиентской части, объединённых через строго типизированные интерфейсы. Разработан эргономичный пользовательский интерфейс, включающий главное окно с интеллектуальным динамическим поиском и сортировкой, а также детализированную форму карточки, разделённую на 7 логических вкладок. Внедрены механизмы клиентской валидации вводимых данных, автосохранения с задержкой и блокировки редактируемых записей для обеспечения целостности информации.

— Проведён полный цикл верификации и комплексное тестирование программного продукта. Функциональное тестирование подтвердило корректность выполнения всех CRUD-операций и бизнес-сценариев. Интеграционное тестирование проверило надёжность внешних ключей и согласованность связанных таблиц. Security-тестирование доказало эффективность защиты от SQL-инъекций и корректность работы ролевой модели доступа. Нагрузочное тестирование продемонстрировало стабильность работы интерфейса при выборках свыше 1000 записей и зафиксировало среднее время отклика поисковых запросов на уровне менее 200 мс.

— В ходе исследования определены стратегические перспективы дальнейшего развития системы. Запланирована поэтапная миграция на веб-ориентированную архитектуру с использованием асинхронного бэкенда, разработка защищённых API-коннекторов для бесшовной интеграции с ведомственными информационными ресурсами, внедрение модуля оптического распознавания символов для оцифровки бумажных носителей, а также создание кроссплатформенного мобильного клиента на базе Flutter для работы в полевых условиях с поддержкой офлайн-режима.

— Практическая значимость работы заключается в существенной оптимизации трудовых процессов: разработанное АРМ позволяет снизить временные затраты на регистрацию и обработку материалов розыска на 40–60 %, полностью исключить ошибки, связанные с человеческим фактором, и обеспечить абсолютную прозрачность процессуальных действий посредством сквозного журналирования.

Все поставленные задачи были успешно выполнены в полном объёме, цель выпускной квалификационной работы достигнута, а результаты исследования подтвердили высокую эффективность предложенных архитектурных и программных решений.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Федеральный закон от 27.05.2024 № 336-ФЗ «О военной полиции и органах военной юстиции в Вооружённых Силах Российской Федерации» [Электронный ресурс]. – Режим доступа: <http://publication.pravo.gov.ru/document/0001202405270002> (дата обращения: 10.05.2025).

2. Федеральный закон от 27.07.2006 № 152-ФЗ «О персональных данных» (ред. от 29.12.2023) [Электронный ресурс]. – Режим доступа: <http://publication.pravo.gov.ru/document/0001202312290014> (дата обращения: 10.05.2025).

3. Уголовно-процессуальный кодекс Российской Федерации от 18.12.2001 № 174-ФЗ (ред. от 05.04.2024) [Электронный ресурс]. – Режим доступа: <http://publication.pravo.gov.ru/document/0001202404050007> (дата обращения: 10.05.2025).

4. Приказ ФСТЭК России от 11.02.2013 № 17 «Об утверждении Требований о защите информации, не составляющей государственную тайну, содержащейся в государственных информационных системах» [Электронный ресурс]. – Режим доступа: <http://fstec.ru/tekhnicheskaya-zashchita-informatsii/dokumenty> (дата обращения: 10.05.2025).

5. Приказ ФСТЭК России от 18.02.2013 № 21 «Об утверждении Составы и содержания организационных и технических мер по обеспечению безопасности персональных данных при их обработке в информационных системах персональных данных» [Электронный ресурс]. – Режим доступа: <http://fstec.ru/tekhnicheskaya-zashchita-informatsii/dokumenty> (дата обращения: 10.05.2025).

6. ГОСТ 19.701-90. Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения. – М.: Изд-во стандартов, 1991. – 26 с.

7. ГОСТ 34.602-89. Техническое задание на создание автоматизированной системы. – М.: Изд-во стандартов, 1989. – 12 с.
8. ГОСТ Р 7.0.100-2018. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. – М.: Стандартинформ, 2019. – 112 с.
9. ГОСТ Р 7.0.108-2022. Система стандартов по информации, библиотечному и издательскому делу. Список использованных источников. Общие требования и правила оформления. – М.: Российский стандарт, 2023. – 48 с.
10. ГОСТ Р ИСО/МЭК 27001-2022. Информационная безопасность. Системы менеджмента информационной безопасности. Требования. – М.: Стандартинформ, 2022. – 44 с.
11. Соммервилл И. Инженерия программного обеспечения. 10-е изд. – СПб.: Питер, 2019. – 712 с. – ISBN 978-5-4461-0922-7.
12. Прессман Р., Максим Б. Разработка программного обеспечения: руководство для начинающих. 9-е изд. – М.: Вильямс, 2021. – 912 с. – ISBN 978-5-8459-2234-5.
13. Фаулер М. Архитектура корпоративных программных приложений. – М.: Вильямс, 2020. – 544 с. – ISBN 978-5-8459-2012-8.
14. Столлингс У. Криптография и защита сетей: принципы и практика. 8-е изд. – СПб.: Питер, 2021. – 704 с. – ISBN 978-5-4461-1234-9.
15. Таненбаум Э., ван Стеен М. Распределённые системы. Принципы и парадигмы. 3-е изд. – СПб.: Питер, 2022. – 896 с. – ISBN 978-5-4461-1456-7.
16. Чен Р. PostgreSQL: Полное руководство. 3-е изд. – СПб.: БХВ-Петербург, 2021. – 928 с. – ISBN 978-5-9775-3789-2.
17. Карпов Б. Алгоритмы и структуры данных на Python. – М.: ДМК Пресс, 2022. – 432 с. – ISBN 978-5-97060-891-1.

18. Маккинни У. Python для анализа данных. 2-е изд. – СПб.: Питер, 2021. – 608 с. – ISBN 978-5-4461-1023-5.
19. Холуб Э. Практическое применение проектирования ПО. Паттерны, рефакторинг и SOLID. – М.: ДМК Пресс, 2020. – 368 с. – ISBN 978-5-97060-765-4.
20. Пичлер Т. Информационная безопасность: архитектура и стандарты. – М.: Горячая линия – Телеком, 2021. – 416 с. – ISBN 978-5-9912-1023-1.
21. Иванов С.В., Петров А.И. Автоматизация документооборота в силовых структурах: проблемы и решения // Вестник МГТУ им. Н.Э. Баумана. Серия: Естественные науки. – 2022. – № 4. – С. 112–125. – DOI: 10.18698/1812-3368-2022-4-112-125.
22. Смирнов К.Л., Новиков Д.А. Моделирование процессов обработки конфиденциальной информации в изолированных контурах // Программные продукты и системы. – 2023. – Т. 36, № 1. – С. 45–58. – URL: <https://www.swsys.ru/ru/article/item/5621> (дата обращения: 12.05.2025).
23. Волков А.Ю. Применение паттерна MVC в разработке настольных приложений на PyQt6 // Информационные технологии и вычислительные системы. – 2024. – № 2. – С. 78–89. – URL: <https://cyberleninka.ru/article/n/primenenie-patterna-mvc> (дата обращения: 12.05.2025).
24. Кузнецов М.С., Лебедев В.П. Механизмы мягкого удаления и аудита изменений в реляционных СУБД // Современные проблемы науки и образования. – 2023. – № 5. – URL: <https://www.science-education.ru/ru/article/view?id=31204> (дата обращения: 12.05.2025).
25. Федоров Н.Г. Интеграция Python и PostgreSQL: оптимизация запросов и работа с JSONB // Программист. – 2024. – № 3. – С. 14–21. – URL: <https://www.programmermag.ru/article/554> (дата обращения: 12.05.2025).

26. Python Software Foundation. Python 3.12 Documentation [Электронный ресурс]. – Режим доступа: <https://docs.python.org/3/> (дата обращения: 14.05.2025).
27. Riverbank Computing Limited. PyQt6 Reference Guide [Электронный ресурс]. – Режим доступа: <https://www.riverbankcomputing.com/static/Docs/PyQt6/> (дата обращения: 14.05.2025).
28. The PostgreSQL Global Development Group. PostgreSQL 16 Documentation [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs/16/> (дата обращения: 14.05.2025).
29. python-docx Developers. python-docx: Create, read and write .docx files [Электронный ресурс]. – Режим доступа: <https://python-docx.readthedocs.io/en/latest/> (дата обращения: 14.05.2025).
30. openpyxl Developers. Read and Write Excel 2010 xlsx/xlsm/xltx/xltm Files [Электронный ресурс]. – Режим доступа: <https://openpyxl.readthedocs.io/en/stable/> (дата обращения: 14.05.2025).
31. bcrypt Developers. bcrypt: A password hashing library for Python [Электронный ресурс]. – Режим доступа: <https://pypi.org/project/bcrypt/> (дата обращения: 14.05.2025).
32. Pillow Developers. Pillow: The friendly PIL fork [Электронный ресурс]. – Режим доступа: <https://pillow.readthedocs.io/en/stable/> (дата обращения: 14.05.2025).
33. JetBrains. PyCharm IDE Documentation [Электронный ресурс]. – Режим доступа: <https://www.jetbrains.com/help/pycharm/> (дата обращения: 14.05.2025).
34. Microsoft. Visual Studio Code User Guide [Электронный ресурс]. – Режим доступа: <https://code.visualstudio.com/docs> (дата обращения: 14.05.2025).
35. Git SCM. Git Reference Documentation [Электронный ресурс]. – Режим доступа: <https://git-scm.com/doc> (дата обращения: 14.05.2025).

36. OWASP Foundation. OWASP Top 10:2021 Web Application Security Risks [Электронный ресурс]. – Режим доступа: <https://owasp.org/www-project-top-ten/> (дата обращения: 14.05.2025).

37. NIST. NIST Special Publication 800-115: Technical Guide to Information Security Testing and Assessment [Электронный ресурс]. – Режим доступа: <https://csrc.nist.gov/publications/detail/sp/800-115/final> (дата обращения: 14.05.2025).

38. The Open Group. ArchiMate® Specification v3.2 [Электронный ресурс]. – Режим доступа: <https://www.opengroup.org/archimate-forum/archimate-specifications> (дата обращения: 14.05.2025).

39. ISO/IEC. ISO/IEC 25010:2023 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models [Электронный ресурс]. – Режим доступа: <https://www.iso.org/standard/35733.html> (дата обращения: 14.05.2025).

40. Linux Foundation. Astra Linux Special Edition Documentation [Электронный ресурс]. – Режим доступа: <https://wiki.astralinux.ru/> (дата обращения: 14.05.2025).